

LUCRAREA NR. 1

MEDII DE TRANSMISIE BAZATE PE CUPRU ȘI CABLAREA UTP

1. Scopul lucrării

Obiectivul acestei lucrări este cunoașterea mediilor de transmisie bazate pe cupru și cablarea și testarea cablării UTP.

2. Considerații teoretice

2.1 Cabluri coaxiale și torsadate

În transmisia de date, mediul de transmisie reprezintă calea fizică între emițător și receptor, care trebuie să asigure performanțe superioare exprimate sub forma unor parametri cum ar fi viteza de comunicare, rata de erori a transmisiei, costul, necesarul de amplificare.

Caracteristicile și calitatea unei transmisii de date sunt determinate atât de caracteristicile mediului suport pentru transmisie cât și de cele ale semnalului propagat.

În proiectarea sistemelor de transmisie de date elementele determinante pentru performanțele sistemului sunt:

- *lățimea de bandă* – reprezintă volumul de date transferate pe un canal de comunicație astfel încât dacă ceilalți factori rămân constanți, cu cât lățimea de bandă este mai mare se va obține o rată de transmisie a semnalului mai bună;
- *interferența* – este generată de suprapuneri de semnale în aceeași bandă de frecvență, fapt ce poate genera distorsionarea semnalului. Ecranarea corectă a mediului de transmisie poate determina minimizarea efectelor de acest tip;
- *numărul de receptori* - presupune construirea de legături punct la punct sau partajate.

Parametri electrici principali ai mediilor de transmisie bazate pe cupru sunt:

- *impedanța* – pentru transmisiile de date interesează nu doar valoarea impedanței la o frecvență dată ci și variația ei în funcție de frecvență;
- *viteza de propagare* - reprezintă un procent din viteza luminii;
- *atenuarea* – de acest parametru depinde comportarea la frecvențe înalte a canalului. Această valoare crește proporțional cu lungimea cablului.
- *diafonia* - este măsura influenței produse de un cablu asupra altui cablu aflat în vecinătate.

Cablul coaxial este mediul de transmisie cel mai versatil, utilizat într-o mare varietate de aplicații, de la transmisia telefonică pe distanțe lungi, rețele locale de calculatoare, până la distribuția TV pentru conectarea diverselor dispozitive. Acesta este un mediu ce permite operarea pe un spectru larg de frecvențe. Tendința actuală este de a înlocui acest tip de cablu deoarece caracterizează un mediu de transmisie cu acces partajat, imposibil de utilizat în rețele de mare viteză cu legături de tip full-duplex și elemente de comutare rapide. Performanțele sale au fost atinse și depășite pentru distanțe scurte de cablul torsadat, iar pe

distanțe lungi de fibra optică. Cablul conține un miez de cupru izolat de al doilea conductor exterior, realizat sub forma unui ecran dintr-o țesătură de fire subțiri. Principalele caracteristici ale cablului coaxial sunt:

- permite transmisie de semnale digitale și analogice;
- datorită modului de construcție concentrică, este rezistent la interferență magnetică.

Constrângerile principale legate de performanțe se referă la atenuarea introdusă, zgomotele intermodulare și încălzirea sa.

Cablul coaxial folosit în rețele locale de calculatoare are impedanța de 50 Ohmi și este de 2 tipuri:

- *cablul coaxial subțire* (RG58 în rețele IEEE 802.3 de tip **10BASE2**) este cel mai răspândit și utilizat pentru instalări de interior datorită unui raport preț/performanță bun;
- *cablul coaxial gros* (RG213 în rețele IEEE 802.3 tip **10BASE5**) este utilizat pentru instalări de exterior datorită rezistenței mecanice mai mari și limitei de lungime mai bune.

Conectarea calculatoarelor la cablul coaxial se realizează prin două metode: folosind joncțiuni T sau conectori speciali numiți *conector vampir* plasați în cadrul unui dispozitiv numit transceiver, care permit înfigerea lor în cablu fără a fi necesară tăierea acestuia. Conectorul penetrează stratul izolator realizând contactul direct cu stratul conductor. Conexiunea de la transceiver la placa de rețea se realizează printr-un cablu de transceiver care se conectează la portul AUI (Attachment Unit Interface). Pentru transmisia analogică pe distanțe mari sunt necesare amplificatoare de semnal, iar pentru semnalele digitale sunt necesare repetitoare, standardele specificând exact distanța de amplasare a acestora. În cazul cablului coaxial subțire, distanța maximă este 185m iar în cazul cablului coaxial gros, distanța maximă este 500m.

Cablul torsadat (*twisted pair - TP*) sau cablul cu perechi de fire de cupru răsucite, având un înveliș comun (cu sau fără ecranare), reprezintă tipul de cablu uzual folosit în rețele locale de calculatoare și telefonie. Răsucirea firelor are drept scop reducerea distorsiunilor magnetice, a interferențelor între perechile adiacente de cablu. Acest cablu acționează asemeni unei singure legături de comunicație. Pentru cablurile cu mai multe perechi de fire răsucite, pașii de răsucire trebuie să fie diferiți pentru fiecare pereche astfel încât diafonia între perechi să fie minimă. Datorită progreselor realizate în tehnologia de realizare a cablurilor TP, acestea pot fi utilizate într-o gamă foarte largă de frecvențe permițând transmisii de date de până la 125 Mbps, iar în rețele Gigabit oferind pe distanțe scurte performanțe comparabile cu fibra optică. Cablul TP reprezintă mediul de transmisie pentru semnale analogice și digitale utilizat de obicei în telefonie și rețele locale de calculatoare.

Există trei tipuri de cabluri TP folosite în rețele de calculatoare, fiecare având patru perechi de fire torsadate și permițând o distanță maximă de 100m:

- *cabluri ecranate STP* (Shielded Twisted Pair), care au ecranare pentru fiecare pereche de fire în parte și pentru tot cablul. Aceste cabluri au o impedanță de 150 Ohmi, fiind folosite în rețele de 10 și 100Mbps;
- *cabluri ScTP* (Screened UTP) sau *FTP* (Foiled Twisted Pair) care au doar ecranare pentru tot cablul. Aceste cabluri au o impedanță de 100 sau 120 Ohmi, fiind folosite în rețele cu viteze de 10 și 100Mbps.

- *cabluri neecranate UTP* (Unshielded Twisted Pair) care nu au ecranare. Este cel mai utilizat tip de cablu și are impedența de 100 Ohmi, fiind folosit în rețele cu viteze de 10, 100 și 1000Mbps.

Categoriile de cabluri torsadate folosite în transmisiile de date se diferențiază în funcție de utilizările pe care le suportă:

- *Categoria 1* – desemnează cablurile pentru telefonie clasică, analogică;
- *Categoria 2* – descrie cablurile pentru telefonie digitală și analogică, dar care nu oferă servicii de transmitere date la viteze superioare;
- *Categoria 3* - cabluri care pot fi folosite pentru LAN-uri de viteze obișnuite, de tip Ethernet 802.3, cu viteze de până la 16Mbps și a rețelor Token Ring la 4Mbps;
- *Categoria 4* – definește cabluri cu performanțe ridicate, ale căror caracteristici de transmisie depășesc 20Mbps;
- *Categoria 5* – descrie cabluri potrivite pentru rețele Fast Ethernet care rulează la viteza de 100Mbps. Această categorie de cabluri este frecvent utilizată în cablări, datorită performanțelor ridicate pe care le oferă.
- *Categoria 5e* – descrie cabluri potrivite pentru rețele Gigabit Ethernet care rulează la viteza de 1Gbps. Și această categorie de cabluri este frecvent utilizată în cablări, datorită performanțelor ridicate pe care le oferă.
- *Categoria 6* – permite transmisii de date de până la 155Mbps;
- *Categoria 7* – permite transmisii de date de până la 1Gbps.

Toate aceste clasificări nu se referă doar la cabluri ci și la întreaga conectică asociată: mufe, prize, patch panel-uri etc. Cablul torsadat permite realizarea de legături punct la punct, realizând topologii diverse ale rețelei de tip stea sau stea extinsă. Acest fapt oferă performanțe deosebite comparativ cu topologia de tip magistrală, realizată cu precădere folosind cablul coaxial.

2.2 Cablarea UTP

La cablarea UTP a rețelelor Ethernet și Fast Ethernet perechea de fire 1-2 se folosește pentru transmisie, iar perechea 3-6 pentru recepție. Acest tip de dispunere a firelor se numește MDI (Media Dependent Interface) sau dispunere normală (straight cable). În mod uzual firele se conectează după aceeași regulă la mufele din cele două capete ale cablului, caz în care cablul (patch cord) se numește drept sau direct. În unele cazuri speciale trebuie inversată recepția cu transmisia pentru a face posibilă comunicarea, caz în care cablul (patch cord) se numește inversor. La cablarea UTP a rețelelor Gigabit Ethernet toate patru perechile de fire se folosesc atât pentru transmisie cât și pentru recepție. Cablurile UTP conțin patru perechi de fire torsadate fiecare pereche identificându-se printr-o culoare: albastru, oranj, verde și maro. Fiecare pereche conține un fir colorat și un fir alb combinat cu culoarea respectivă. Mufele folosite pentru acest cablu sunt mufe tată de tip RJ-45 conținând 8 pini corespunzători celor 8 fire. Privit din față, pinii sunt numerotați de la 1 în dreapta la 8 în stânga. Modul de conectare a firelor la pini determină tipul cablului. Există două standarde de conectare a firelor la mufa RJ-45: EIA/TIA-T568-A și EIA/TIA-T568-B. Aceste conectări sunt prezentate mai jos.

Tabelul 1.1 EIA/TIA-T568-A

Pin#	Pereche#	Funcție	Culoare fir	Folosit cu 10/100BASE-T	Folosit cu 100BASE-T4 și 1000 BASE-TX
1	3	Transmisie+	Alb/Verde	Da	Da
2	3	Transmisie-	Verde	Da	Da
3	2	Recepție+	Alb/Oranj	Da	Da
4	1	Nefolosit	Albastru	Nu	Da
5	1	Nefolosit	Alb/Albastru	Nu	Da
6	2	Recepție-	Oranj	Da	Da
7	4	Nefolosit	Alb/Maro	Nu	Da
8	4	Nefolosit	Maro	Nu	Da

Tabelul 1.2 EIA/TIA-T568-B

Pin#	Pereche#	Funcție	Culoare fir	Folosit cu 10/100BASE-T	Folosit cu 100BASE-T4 și 1000 BASE-TX
1	2	Transmisie+	Alb/Oranj	Da	Da
2	2	Transmisie-	Oranj	Da	Da
3	3	Recepție+	Alb/Verde	Da	Da
4	1	Nefolosit	Albastru	Nu	Da
5	1	Nefolosit	Alb/Albastru	Nu	Da
6	3	Recepție-	Verde	Da	Da
7	4	Nefolosit	Alb/Maro	Nu	Da
8	4	Nefolosit	Maro	Nu	Da

Așadar, pentru a obține un cablu drept sau direct ambele capete ale cablului trebuie să mufate după același standard și pentru a obține un cablu inversor fiecare capăt al cablului trebuie mufat după câte un standard.

La cablarea UTP se folosesc atât dispozitive pasive cât și dispozitive active. Dispozitivele pasive nu sunt alimentate de la o sursă de tensiune în vreme ce dispozitivele active necesită alimentare. Cele mai importante dispozitive pasive sunt: jack RJ-45, priza și patch panel-ul. La nivel 1, cele mai importante dispozitive active sunt hub-ul și transceiver-ul. Un jack RJ-45 este un dispozitiv cu opt conductori în care se conectează mufa RJ-45. Acesta intră în componența prizelor și a patch panel-urilor. În prize se conectează calculatoarele cu ajutorul patch cord-urilor. Prizele sunt legate la patch panel care se află în dulapul de distribuție. Cu ajutorul unui patch cord se conectează patch panel-ul la hub care se afla localizat tot în dulapul de distribuție. Hub-ul este un repetor multiport. Transceiver-ul este un dispozitiv bidirecțional care recepționează semnalele de la un tip de interfață, le convertește în semnale specifice altui tip de interfață și le transmite unei interfețe de acel tip.

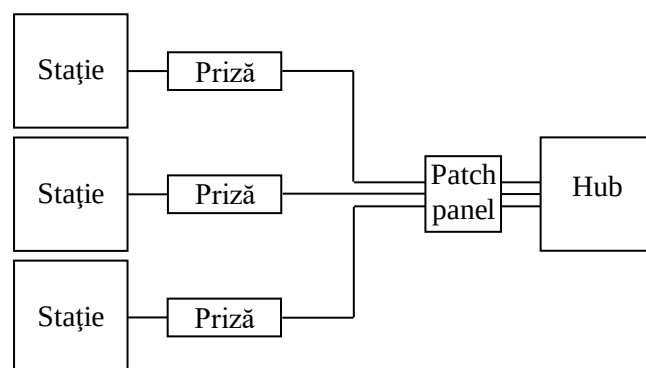
3. Desfășurarea lucrării

3.1 Mufarea și testarea cablurilor UTP

Se vor mufa și testa mai multe cabluri drepte sau directe. Se vor mufa și testa mai multe cabluri inversoare.

3.2 Cablarea și testarea rețelelor

Folosindu-se un cablu inversor se va testa conectivitatea dintre două calculatoare. Cablați și testați conectivitatea rețelei prezentată în figură.



LUCRAREA NR. 2

FIBRE OPTICE ȘI COMPONENTE OPTICE

1. Scopul lucrării

Obiectivul acestei lucrări este cunoașterea fibrelor și componentelor optice și calculul bugetului optic.

2. Considerații teoretice

2.1 Fibre și componente optice

Odată cu scăderea accentuată a prețului **fibrei optice**, și a echipamentelor de comunicație corespunzătoare, aceasta a devenit mediul preferat pentru noile conexiuni de exterior, de mare viteză, folosindu-se uneori și în interiorul clădirilor.

Fibra optică conține un fir foarte subțire numit miez, de ordinul zecilor de microni (μ), care constituie un ghid de undă pentru lumină, realizat din sticlă obținută dintr-o combinație de dioxid de siliciu și alte elemente. În cazul fibrei multimod indicele de refracție al sticlei scade de la centru spre exterior. Acest tip de sticlă se numește sticlă cu index variabil sau gradat și permite diferitelor moduri luminoase să ajunga la receptor în același moment. În cazul fibrei monomod indicele de refracție al sticlei este constant. Acest tip de sticlă se numește sticlă cu index pas. Miezul este învelit de un material realizat din dioxid de siliciu având un indice de refracție mai mic decât al miezului numit imbrăcămintă. Pentru a proteja imbrăcămintea, aceasta este învelită într-un material plastic. Acest înveliș se numește protecție și este învelit la rândul său de un material întăritor, de obicei Kevlar, care conferă rezistență fibrei în momentul instalării. Ultimul înveliș este mantaua care protejează fibra împotriva materialelor abrazive, a solventilor și a altor factori. Culoarea mantalei în cazul fibrei optice multimod este de obicei portocaliu și în cazul fibrei optice monomod este de obicei galben. Fiecare cablu de fibră optică este compus din două fibre învelite separat, o fibră fiind folosită pentru transmisie și alta pentru recepție, asigurându-se în acest mod o legătură full-duplex. Un cablu de fibră optică poate conține de la 2 până la 48 de fibre separate învelite într-o teacă. Figura 2.1 prezintă o secțiune transversală prin fibra optică.

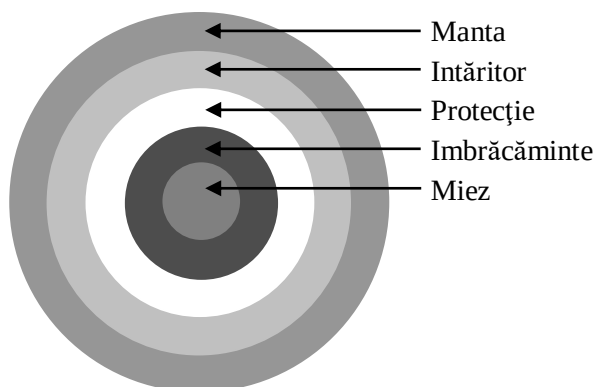


Figura 2.1 Secțiune transversală prin fibra optică

Pentru ca semnalul luminos să fie reflectat fără pierderi trebuie îndeplinite următoarele două condiții:

- fibra optică trebuie să aibă un indice de refracție mai mare decât materialul care o înconjoară;
- unghiul de incidență al semnalului luminos trebuie să fie mai mare decât unghiul critic al fibrei și al materialului care o înconjoară. Unghiul de incidență al semnalului luminos poate fi controlat cu ajutorul următorilor doi factori:
 - apertura numerică a fibrei este gama unghiurilor semnalului luminos pentru care reflexia este totală;
 - modurile reprezintă căile pe care semnalul luminos le poate urma.

Miezul fibrei **multimod** are diametrul suficient de mare încât să permită mai multe moduri (mai multe căi) semnalului luminos. Cablurile de fibră optică multimod standard folosesc miezul cu diametrul de $62,5\mu$ sau 50μ și învelișul cu diametrul de 125μ . Acestea sunt de obicei referite ca și fibre optice de $62.5/125\mu$ sau $50/125\mu$. De obicei, sursele de lumină folosite cu fibra multimod sunt Infrared Light Emitting Diode (LED) sau Vertical Cavity Surface Emitting Lasers (VCSEL). LED-urile sunt mai ieftine și necesită mai puține măsuri de siguranță decât laserele. Dezavantajul LED-urilor este că nu pot transmite semnalele luminoase la distanțe la fel de mari ca și laserele. Fibra multimod de $62.5/125$ poate transmite date la distanțe de până la 2000m. Pierdere pe km de fibră optică multimod este specificată de producător și are valori cuprinse în intervalul 1-2dB/km.

Miezul fibrei **monomod** are diametrul suficient de mic încât să permită doar un singur mod (o singură cale) semnalului luminos, acesta fiind transmis în linie dreaptă prin mijlocul miezului. Cablurile de fibră optică monomod folosesc miezul cu diametrul între 8μ și 10μ . Cele mai folosite fibre optice monomod au diametrul de 9μ și învelișul cu diametrul de 125μ . Acestea sunt de obicei referite ca și fibre optice de $9/125\mu$. Sursa de lumină folosită cu fibra monomod este laserul infraroșu. Se recomandă precauție atunci când se folosește laserul ca și sursă de lumină deoarece acesta poate afecta ochii. Fibra monomod poate transmite date la distanțe de până la 3000m. Pierdere pe km de fibră optică monomod este specificată de producător și are valoarea de aproximativ 0,5dB/km.

În fibra optică lumina suferă pe lângă propagare două fenomene principale: atenuare și dispersie. Atenuarea sau absorbția se datorează în principal prezenței ionilor hidroxil -OH și a diferiților ioni de metale. Lumina poate fi de asemenea împrăștiată de microcristale, mai mici decât lungimea de undă, care se formează la răcirea sticlei. Atenuarea limitează utilizarea fibrei

optice în lungime. Dispersia sau lărgirea lății impulsurilor se datorează în fibra multimod lungimii diferite pe care o au diferitele moduri. O altă dispersie cea cromatică este datorată variației indicelui de refracție în funcție de culoarea sau lungimea de undă a luminii. Dispersia limitează utilizarea fibrei optice în frecvență sau lărgime de bandă. Cele două limitări înmulțite caracterizează cel mai corect o fibră optică. Valori de 20MHz-km se obțin pentru fibra cu index pas, de 1GHz-km pentru cea cu index variabil și de 1000GHz-km pentru cea monomod la care nu există dispersie modală.

Transmițătoarele pentru fibra optică convertesc semnalele electrice în pulsuri luminoase echivalente. Există două tipuri de surse de lumină folosite de transmițătoarele pentru fibra optică:

- LED-ul care produce lumina în infraroșu având lungimea de undă de 850nm sau 1310nm. Acestea sunt folosite cu fibre multimod. Cuplarea la fibra optică poate fi îmbunătățită prin utilizarea unei lentile sferice;
- dioda semiconductoră LASER care produce lumina în infraroșu având lungimea de undă de 1310nm sau 1550nm. Acestea sunt folosite cu fibre multimod sau monomod.

Există două tipuri constructive de bază pentru LED-uri: cu emisie pe suprafață și cu emisie pe muchie. La LED cu emisie pe suprafață, emisia luminii are loc perpendicular pe planul joncțiunii printr-un strat subțire transparent. Acestea emit într-un spectru geometric radial. La LED cu emisie pe muchie lumina este emisă într-un plan paralel cu joncțiunea la muchia semiconductorului. Materialele cel mai des utilizate sunt compuși III-V ca GaAs sau $\text{Al}_x\text{Ga}_{1-x}\text{As}$ pentru lungimi de undă de 0,8-0,9 μm și $\text{Ga}_x\text{In}_{1-x}\text{P}_y\text{As}_{1-y}$ pentru lungimi de undă de 1,3-1,6 μm . Spectrul de emisie a unui LED este cuprins între 25-40 μm pentru lungimi de undă mici și 50-100 μm pentru lungimi de undă mai mari.

Diodele semiconductoră LASER, diode laser (LD), se obțin prin introducerea unui LED într-o cavitate rezonantă optic. Efectul de LASER apare numai la existența unui curent direct suficient de mare pentru a se realiza o inversare de populații a electronilor și a golurilor din cele două benzi energetice de conducție și de valență. Valoarea de curent de la care apare acest efect se numește curent limită. Sub acest curent dispozitivul se comportă ca un LED obișnuit. Deoarece lumina emisă de un laser este mult mai coerentă decât cea emisă de un LED, eficiența de cuplare la fibra optică este superioară. De asemenea puterea optică captată de la un laser este mai mare decât cea emisă de LED.

O analiză comparată între cele două tipuri de emițătoare este clar în favoarea LD prin posibilitatea de utilizare la frecvențe mai mari, spectru mai restrâns și în favoarea LED ca preț și stabilitate mai mare a puterii în raport cu temperatura.

Timpul de viață al ambelor dispozitive este egal și de ordinul a 10 milioane ore.

Receptoarele pentru fibra optică convertesc pulsurile luminoase în semnale electrice echivalente. Dispozitivele semiconductoră folosite de obicei de receptoarele pentru fibra optică se clasifică în două tipuri: simple și cu câștig intern. Primele se mai numesc și fotodiode PIN după tipul de dopare (p intrinsec și n) iar cea de a doua categorie se numește APD (Avalanche Photo-Diodes). Aceste dispozitive sunt sensibile la lungimile de undă ale luminii de 850, 1310 și 1550nm, lungimi de undă folosite de transmițătoarele pentru fibra optică. Ca materiale semiconductoră sunt folosite Si pentru lungimi de undă de 800-900 nm și Ge sau InGaAsP pentru 1300 și 1500 nm. Si are sensibilitate optimă doar într-o zonă de frecvențe redusă pe când Ge are un curent de întineric apreciabil și este mai sensibil la zgomot. Din acest motiv ultima variantă este cea mai bună dar necesită o tehnologie de fabricație mai sofisticată și în consecință are și un preț mai mare.

Pentru a conecta fibre, cu scopul realizării unei fibre mai lungi se folosesc **joncțiuni**. Joncțiunile sunt de două tipuri: mecanice și de fuziune. Atenuările introduse sunt de ordinul a 0.5dB la joncțiunile mecanice și de 0.1dB la cele de fuziune. La joncțiunile mecanice cele

două capete de fibră, atent tăiate, curățate și șlefuite sunt prinse într-o montură mecanică rigidă care le fixează una față de cealaltă într-un ansamblu imobil. Joncțiunile de fuziune se execută prin încălzirea aproape până la punctul de topire. În acest moment cele două fibre sunt lipite una de alta și răcite. Aceste operații sunt precedate de operații de tăiere și finisare a capetelor și de aliniere prealabilă a celor două capete de jonctat. Joncțiunile de fuziune refac și rezistența la tragere/ruptură a fibrei la aproximativ 90% din cea inițială.

Conectorii pentru fibra optică permit conectarea fibrelor la porturi. Cei mai folosiți conectori sunt SC (Subscriber Connector) pentru fibre optice multimod și ST (Straight Tip) pentru fibre optice monomod. Atenuarea introdusă de un conector optic, chiar de calitate superioară este mai mare decât cea introdusă de o joncțiune, având valori de aproximativ 1dB. Conectorii sunt echipamente mecanice de mare precizie și de obicei un capăt al fibrei se află în conector iar unul este liber. În acest caz atașarea unui conector se reduce la execuția unei joncțiuni. O astfel de soluție este de obicei mai avantajoasă decât montarea unui conector direct pe capătul fibrei deoarece conectorii prefabricați asigură o precizie de montare mult mai mare. Dacă fibra optică este terminată într-un terminator de fibră optică pentru redistribuire acest conector de capăt se mai numește și pig-tail și este de tipul prefabricat. O categorie specială de conectori o constituie cordonurile optice de distribuție sau legătură. Acestea sunt fibre optice speciale cu conectori la ambele capete care permit raze de curbura a fibrei mici de ordinul 2,5-5 cm. Culoarea acestora este galben pentru fibra monomod și portocaliu pentru fibra multimod.

Repetoarele sunt amplificatoare optice care receptionează semnalele luminoase atenuate ca urmare a distanței parcurse prin fibra optică, refac forma, puterea și parametri de timp a acestor semnale și le transmit mai departe.

Patch panel-urile pentru fibră sunt similare patch panel-urilor pentru cablul de cupru mărind flexibilitatea rețelilor optice.

Instalarea incorectă a fibrelor optice are ca și rezultat creșterea atenuării semnalului optic. Întinderea sau curbarea exagerată a fibrei optice poate cauza mici fisuri ale miezului care vor dispersa semnalul luminos. Curbarea exagerată a fibrei optice poate avea ca urmare scăderea unghiului incident al semnalului luminos sub unghiul critic de reflexie totală. Pentru instalarea conectorilor capetele fibrei trebuie tăiate și finisate. După instalare, capetele fibrelor optice, conectorii și porturile de fibră trebuie păstrate curate pentru a nu introduce atenuări. Înaintea folosirii cablurilor de fibră optică, trebuie testată atenuarea introdusă de acestea. La proiectarea unei legături pe fibră optică, trebuie calculată pierderea puterii semnalului care poate fi tolerată. Aceasta se numește bugetul de pierdere a legăturii optice. Pierderea puterii se măsoară în decibeli (dB).

Pentru testarea unei legături prin fibră optică există mai multe procedee: procedeul de măsurare a puterii optice la ieșire, procedeul OTDR și testul BER de rată a erorilor.

Procedeul de măsurare a puterii optice la ieșire determină pierderile de putere prin legătura optică măsurând puterea la ieșire la o putere de intrare cunoscută. Unitatea de măsură pentru puteri optice este miliwattul (mW) însă din considerente practice se utilizează o altă unitate de măsură care măsoară câștigul (G) sau pierderea (L) într-un sistem și anume deciBell-ul (dB).

Procedeul OTDR Optical Time Domain Reflectometer este procedeul prin care se pot vizualiza caracteristicile de atenuare ale unei fibre optice precum și lungimea acesteia. Acest procedeu este singurul prin care se pot detecta pozițiile întreruperilor în fibra optică. OTDR afișează un grafic care are ca axă x lungimea fibrei și ca axă y atenuarea. Din graficul astfel afișat se pot deduce atenuarea fibrei, calitatea joncțiunilor și a conectorilor. De asemenea se poate determina poziția rupturilor în cablu dacă extern cablul nu este afectat.

Testul BER (Bit Error Rate) este testul final la care se supune o legătură de date prin fibră optică. Acest test sau criteriu arată la câți biți transmiși prin fibră se produce o eroare datorată fibrei. Testul BER trebuie să îndeplinească cerințele impuse de producătorii de echipamente DTE ce se cuplează la fibra optică. Pentru rețele de calculatoare acestea cer să fie mai mici decât 1 bit de eroare la 10^9 biți transmiși sau $BER < 10^{-9}$. Pentru testare este nevoie de un generator de secvențe de bit aleatoare și de o interfață la fibra optică dacă se testează o buclă sau de două dacă se testează o singură fibră. Pentru a avea rezultate semnificative testul trebuie să se desfășoare pe o perioadă suficient de lungă astfel încât să se transmită un număr suficient de mare de biți. Perioade de testare de o zi sau două sunt obișnuite dacă se lucrează la rată de bit mare în utilizarea legăturii prin fibră optică și BER mic. Un numărător poate contoriza automat numărul de erori detectate.

2.2 Calculul bugetului de putere optică

Tabelul 2.1 *Calcul buget de putere optica*

Crt.	Pierdere sau Putere Optică	dB
1.	Pierdere pe km în Fibra Optică ___ dB/km X ___ km fibră	___ dB
2.	Pierdere în Joncțiuni ___ dB/joncțiune X ___ joncțiuni	___ dB
3.	Pierdere în Conectoare ___ dB/conector X ___ conectoare	___ dB
4.	Pierderi pe alte Componente	___ dB
5.	Margine de Eroare	___ dB
6.	Pierdere Totală pe Legătură (1+2+3+4+5)	___ dB
7.	Puterea de Emisie Medie a Emițătorului	___ dB
8.	Puterea Medie Recepționată de Receptor (7-6)	___ dB
9.	Dinamica Receptorului ___ dB la ___ dB	
10.	Sensibilitatea Receptorului la o Rată de Erori dată BER ___	___ dB
11.	Putere Rămasă Disponibilă	___ dB

Observații:

La punctul 3. nu se iau în considerare pierderile de conectare a emițătorului la fibra optică, acestea fiind incluse deja.

Valoarea calculată la punctul 8. trebuie să fie în intervalul de la punctul 9. pentru ca receptorul să funcționeze corect.

Valoarea calculată la punctul 11. trebuie să fie pozitivă pentru a avea o legătură de date optică funcțională.

Marginea de eroare se datorează luării în calcul a unor valori medii pentru toate componentele legăturii. Dispersia acestor valori în jurul valorii medii este cunoscută și se

poate lua o margine de eroare suficient de mare ca aceasta să acopere deviațiile de la medie cu o probabilitate de 99,9% sau mai mare.

Cu cât numărul de elemente este mai mare și cu cât se dorește o probabilitate de acoperire mai mare cu atât se va lua o margine de eroare mai mare.

Puterea de emisie optică a emițătorului este o dată de catalog și conține în ea inclusă și pierderea de conectare la un capăt de fibră optică în cazul în care conectarea se face conform recomandărilor. Puterea este mai mare la diode LASER și mai mică la LED. În cazul utilizării de LASER este nevoie pentru distanțe relativ scurte chiar de un atenuator pentru a nu distruge receptorul.

Dinamica receptorului reprezintă plaja de puteri pe care un receptor le poate transforma în semnal electric fără pierderi de informație.

De asemenea este nevoie de o putere optică minimă necesară pentru îndeplinirea condiției de rată de erori tolerată care pentru rețele de calculatoare se situează la valoarea de 1 bit eronat la un miliard de biți transmiși.

2.3 Exemplu de calcul al bugetului de putere optică

Diametrul fibrei optice: Miez 62.5μm/Înveliș 125μm. Apertura numerică a fibrei NA:0,275. Lungimea de undă a echipamentului optic: 1310μm.

Tabelul 2.2

Crt.	Pierdere sau Putere Optică	dB
1.	Pierdere pe km în Fibră Optică 1,8dB/km X 3,5km fibră	6,3dB
2.	Pierdere în Joncțiuni 0,5dB/joncțiune X 2 joncțiuni	1,0dB
3.	Pierdere în Conectoare 1,0dB/conector X 2 conectoare	2,0dB
4.	Pierderi pe alte Componente	0,0dB
5.	Margine de Eroare	2,0dB
6.	Pierdere totală pe Legătură (1+2+3+4+5)	11,3dB
7.	Puterea de Emisie Medie a Emițătorului	-10,0dB
8.	Puterea Medie Recepționată de Receptor (7-6)	-21,3dB
9.	Dinamica Receptorului -10,0dB la -30,0dB	
10.	Sensibilitatea Receptorului la o Rată de Erori dată BER 10^{-9}	-26,0dB
11.	Putere Rămasă Disponibilă (8-10)	+4,7dB

Puterea ajunsă la receptor se încadrează în dinamica receptorului, ceea ce face posibilă funcționarea sa, iar puterea rămasă disponibilă este pozitivă ceea ce ne asigură de o legătură viabilă.

Trebuie ținut cont și de faptul că în cursul vieții legăturii pot apare fenomene de îmbătrânire a materialelor, care duc la creșterea pierderilor de putere, precum și de faptul că fibra optică poate fi ruptă accidental și trebuie joncționată.

Un calcul făcut la limită periclitează durata de exploatare a unei legături prin fibră optică.

2.4 Analiza mediilor fizice

În analiza mediilor fizice se pot alege mai mulți factori de performanță cum ar fi: viteza de transfer, lărgimea de bandă, fiabilitatea sau rata de erori, durata de exploatare, durata medie între două defecte, toleranță la defecte, costuri directe, costuri indirecte, costul per port sau echipament conectat, costul per lărgime de bandă sau costul total per port per lărgime de bandă. Lărgimea de bandă, L_b este un factor de performanță intrinsec fiecărui mediu. Fiabilitatea sau rata de erori, F , este tot un factor de performanță intrinsec a fiecărui mediu și reprezintă raportul dintre numărul de biți transmiși eronat față de numărul total de biți transmiși. Durata de exploatare, D_e , este durata de timp după care mediul trebuie înlocuit el suferind fenomenul de îmbătrânire. Durata medie între două defecțiuni, DMDD, este timpul mediu statistic între două defectări succesive ale mediului pe perioada normată de viață. Toleranța la defecte, T_d , este un factor de performanță indus asupra mediului fizic de tehnologia și arhitectura rețelei utilizate, în multe cazuri însă un anumit mediu nu permite realizarea unei arhitecturi tolerante la erori sau doar a uneia limitate. Costurile directe, C_d , sunt reprezentate de costul efectiv al mediului împreună cu conectori, materialele auxiliare necesare pozării corecte a acestuia și costul manoperei pentru realizarea mediului de comunicație și a testării mediului astfel realizat. Costul per port, C_p , este un factor sintetic care are o valoare de decizie mai mare, fiind un criteriu de decizie global și reflectând costurile totale pentru realizarea infrastructurii fizice raportat la numărul total de porturi sau echipamente conectate. Costul per port per viteză de transfer, C_{pv} , este un factor de performanță mult mai util și care ușurează luarea unei decizii corecte în realizarea unei rețele locale de calculatoare el incluzând și posibilități de extensie viitoare fără a fi necesară schimbarea mediului. Costul total per port per viteză, C_{tpv} , este un factor de performanță complex care caracterizează o rețea locală de calculatoare la nivel global incluzând de asemenea și costurile de echipamente sau tehnologie. O caracterizare din prisma factorilor de performanță mai sus menționați a mediilor fizice de comunicație prezentate anterior se află sintetizată în următorul tabel. Factorii de performanță, în special cei de tip cost, vor fi clasificați relativ fără a da valori absolute care pot fi alterate foarte rapid în timp.

Tabelul 2.3

Mediu	L_b Mbps	Fiabilitate	D_e ani	DMDD	T_d	C_d	C_p	C_{pv}	C_{tpv}	Recomandat în utilizare	Continuare utilizare
Coaxial gros	10	Medie	10	luni	Mic	Mic	Mic	Mediu	Mic	NU	Înlocuire imediată
Coaxial subțire	10	Mică	5	zile	Nu	F. mic	F. mic	Mediu	F. mic	Excepțional	Înlocuire
UTP Cat 3,4	10	Medie	15	ani	Da	Mediu	Mic	Mediu	Mediu	NU	Limitat
UTP Cat 5,6,7	>100	Medie	15	ani	Da	Mediu	Mic	Mic	Mic	DA	DA
F.O. multimod	>100	Mare	30	ani	Da	Mare	Mediu	Mediu	Mediu	DA	DA
F.O. monomod	>100	Mare	30	ani	Da	F. mare	F. mare	Mare	Mare	DA	DA

3. Desfășurarea lucrării

3.1 Se vor discuta caracteristicile diferitelor tipuri de fibre și componente optice și aspectele legate de cablarea rețelelor de calculatoare folosindu-se acest mediu de transmisie.

3.2 Se consideră o fibră optică monomod de $9/125\mu$ având lungimea de 2,5km și pierderea egală cu 0,5dB/km, care conectează două echipamente DTE. Atenuarea introdusă de joncțiuni și conectori este egală cu 0,5 și respectiv 1dB. Marginea de eroare luată în considerare este de 3dB. Puterea de emisie medie a emițătorului este de -15dB, sensibilitatea receptorului la o rată de erori dată BER 10^{-9} este de -25dB și dinamica receptorului este în intervalul -10 ÷ -30dB. Să se calculeze bugetul de putere optică.

LUCRAREA NR. 3

CABLAREA STRUCTURATĂ

1. Scopul lucrării

Obiectivul acestei lucrări este cunoașterea cablării structurate, a topologiilor de rețele și a funcțiilor diferitelor dispozitive de rețea.

2. Considerații teoretice

2.1 Cablarea structurată

Există trei topologii standard de rețele și anume topologia magistrală, stea și respectiv inel:

- **topologia magistrală** este cea mai veche și cunoscută metodă de conectare a calculatoarelor în rețea. Datele sunt transmise tuturor calculatoarelor dar sunt acceptate doar de calculatorul destinație, iar oprirea reflectării semnalului se realizează folosind terminatoare. Figura 3.1 (a) prezintă topologia magistrală.
- **topologia stea** a înlocuit în mare măsură vechea topologie magistrală, ea este caracterizată de faptul că există o componentă centrală numită concentrator (hub) prin intermediul căreia semnalele sunt transmise de la un calculator la toate celelalte. Topologia stea oferă resursele și posibilitatea administrării centralizate. Figura 3.1 (b) prezintă topologia stea.
- **topologia inel** conectează calculatoarele printr-un cablu în forma unei bucle și fiecare calculator acționează ca un repetor amplificând semnalul. Figura 3.1 (c) prezintă topologia inel.

În prezent majoritatea topologiilor folosite sunt combinații ale topologiilor stea, inel și magistrală. Topologia magistrală – stea presupune conectarea unor rețele cu topologie stea prin intermediul unor trunchiuri lineare tip magistrală. Probleme de conectivitate apar în situația în care se defectează un concentrator. Topologia inel-stea este cunoscută și sub numele de inel cablat în stea. În acest caz există un concentrator central care leagă celelalte concentratoare la care sunt atașate stațiile.

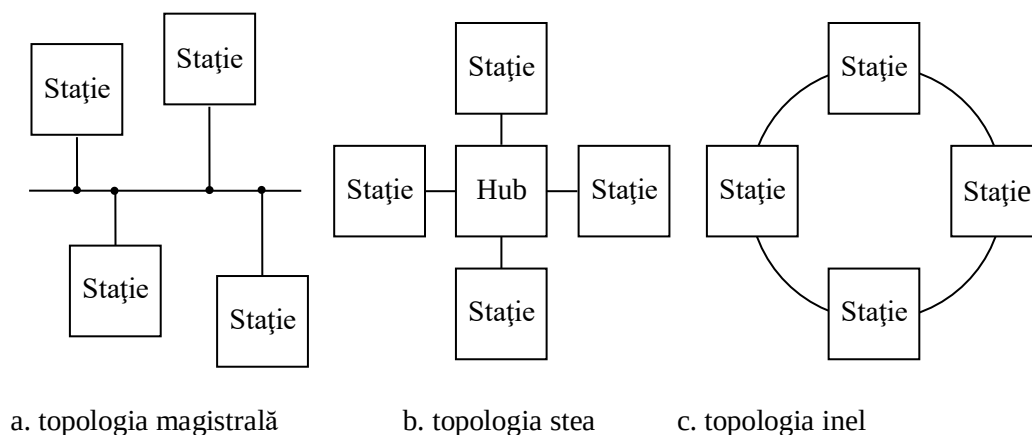


Figura 3.1 Topologii standard de rețele

Sub denumirea generică de elemente active se grupează toate componentele rețelei care necesită alimentare și pot lucra cu semnale electrice, optice sau cu ambele tipuri de semnale.

Plăcile de rețea sunt elemente active de Nivel

2 care asigură conectarea la rețea a calculatoarelor. Fiecare placă de rețea are o adresă proprie MAC pe 48 biți asignată din fabricație. Aceasta adresa este unica pentru fiecare placă de rețea și este compusă din două părți: cei mai semnificativi 24 biți identifică producătorul iar cei mai puțin semnificativi 24 biți sunt asigurați de către acesta. Plăcile de rețea folosite în PC-uri au nevoie de un spațiu de adrese de I/O și de o întrerupere hardware. Întreruperea este activată de fiecare dată când apare un eveniment (de regulă recepție de cadru) care necesită atenția software-ului, iar spațiul de I/O este regiunea de adrese în care sunt accesibili regiștri plăcii (scriși și citiți de către driverul acesteia). De regulă atât întreruperea cât și spațiul de I/O sunt configurabile pentru a se putea evita conflictele cu alte dispozitive.

Depășirea limitărilor de lungime caracteristice cablului coaxial se realizează folosind **repetoare**. Acestea sunt dispozitive simple, conectate la mai multe segmente de rețea care amplifică semnalul care le străbate. Repetoarele operează la nivel fizic (nu au noțiunea de cadru sau pachet transmis în rețea)

și transmit semnalele amplificate pe toate ieșirile lor. Repetoarele pot realiza și conversie de mediu de transmisie între cablu coaxial subțire, coaxial gros și UTP, dar nu permit spre exemplu interconectarea unui segment Ethernet cu segmente Fast Ethernet. Proiectarea rețelelor cu topologii mai complexe se realizează utilizând **hub-uri**. Aceste dispozitive reprezintă funcțional echivalentul repetoarelor, fiind repetoare multiport, au însă un rol mult mai important fiind necesare în orice rețea locală pentru realizarea *topologiei stea*. Hub-urile pot fi plăci instalate în calculatoare, sau echipamente de sine stătătoare, cu un număr variabil de porturi. Hub-urile externe au leduri care indică starea legăturilor, putând astfel deduce dacă este conectat ceva la celălalt capăt și dacă sunt sau nu erori fizice pe firul de conexiune. Huburile pot fi inteligente, caz în care pot fi programate să gestioneze traficul în rețea (dispun de un port de consolă) sau neinteligente caz în care nu dispun de asemenea facilități.

Avantajele pe care le oferă concentratoarele, (respectiv topologiile bazate pe concentratoare) sunt:

- posibilitatea extinderii sau modificării sistemului de cabluri;
- folosirea de porturi diferite, adaptate la diferite tipuri de cabluri;
- posibilitatea monitorizării centralizate a activității și traficului în rețea.

Tipurile de concentratoare sunt:

- **concentratoare active** - care regenerează și transmit semnalul fiind denumite repetoare multiport;
- **concentratoare pasive** - pot fi considerate panourile de cablare sau blocurile de conectare reprezentând doar puncte de conectare fără a amplifica semnalul. De asemenea există și concentratoare hibride care permit utilizarea pentru conectare de tipuri diferite de cabluri.

Hub-urile pot fi conectate la un nivel ierarhic superior, care poate fi spre exemplu un alt hub sau un switch. Portul prin care se realizează cascada trebuie să aibă configurația semnalelor identică cu cea a calculatoarelor. Dacă hub-ul nu are un port pentru care oferă

ambele tipuri de mufe sau un comutator prin intermediul căruia să se realizeze intern cele două configurații pentru unul dintre porturi, atunci inversarea trebuie realizată în cablu.

Odată cu creșterea dimensiunilor rețelei, încep să apară probleme dacă se folosesc numai hub-uri și repetitoare. Limitarea pentru calculatoarele care formează o astfel de rețea este faptul că hub-urile partajează lățimea de bandă, aflându-se într-un singur **domeniu de coliziune**. Pentru rezolvarea acestei probleme se folosesc **bridge-uri**, echipamente ce operează pe Nivelul 2 în ierarhia OSI, și care reprezintă echipamente mult mai complexe decât repetitoarele deoarece realizează o filtrare a cadrelor pe baza adreselor MAC și o separare a domeniilor de coliziune. Bridge-urile nu trimit mai departe cadrele locale uneia din rețelele conectate, ci doar pe cele destinate calculatoarelor din alte rețele. Ele memorează cadrele și realizează retransmisie numai către rețeaua în care se află destinația. La punerea sub tensiune, bridge-urile nu cunosc nimic despre configurația rețelei și despre adresele calculatoarelor conectate la ea, dar învață topologia rețelei pe măsură ce dirijează cadrele. Inițial ele permit trecerea tuturor cadrelor în toate direcțiile. Ulterior, pe măsură ce cadrele trec prin ele, bridge-urile inspectează adresa sursă a fiecărui cadru și își completează tabelele de dirijare, cu adresa stației și portul la care este conectată. Pe baza acestor tabele se decide pe care porturi trebuie retransmise cadrele. Cadrele trimise la adrese de broadcast sau multicast vor fi retransmise mai departe pe toate porturile. **Switchurile** sunt echipamente de Nivel 2 care iau decizii de comutare a cadrelor pe baza adreselor MAC pentru a dirija datele doar pe portul căruia îi corespunde hostul destinație. Acestea pot fi privite ca dispozitive capabile să ofere conectivitatea unui hub și reglarea traficului caracteristică bridge-urilor. Majoritatea switchurilor permit conectarea segmentelor Ethernet cu segmente Fast Ethernet.

Ruterele sunt echipamente de Nivel 3 care rutează pachetele pe baza adresei folosite de protocoalele rutabile (spre exemplu Internet Protocol - IP sau Internetwork Packet Exchange - IPX) cu ajutorul protocoalelor de rutare (spre exemplu Routing Information Protocol RIP, Interior Gateway Routing Protocol – IGRP, Enhanced Interior Gateway Routing Protocol – EIGRP sau Open Shortest Path First - OSPF). Există două categorii principale de rutere: rutere dedicate și rutere realizate din calculatoare cu scop general cu mai multe interfețe. Ruterele realizate din calculatoare au avantajul că sunt

mai ieftine, mai simple și pot fi folosite și pentru alte sarcini. Ruterele dedicate sunt mai eficiente, mai flexibile, au mai multe interfețe și suportă mai multe protocoale și mai multe tipuri de medii de acces. Ruterele dedicate sunt calculatoare specializate pentru sarcina de rutare. Datorită hardware-ului lor specializat și a software-ului puternic optimizat, acestea realizează performanțe superioare. Ele oferă o gamă foarte largă de viteze, interfețe fizice și protocoale de comunicație. De regulă acestea sunt realizate de firme specializate (Cisco, Bay Networks, Proteon etc.) și rulează un sistem de operare propriu, specific, care cuprinde tot software-ul necesar funcționării ruterului. Ruterele dedicate suportă aproape orice mediu de transmisie, utilizat cu orice protocol de comunicație, cu o mulțime de tipuri de mufe și adaptoare.

Având în vedere costurile pe care le implică realizarea unui cablaj sau modificarea acestuia, s-a ajuns la concluzia că este mai bine ca, un cablaj odată realizat să rămână în folosință cât mai multă vreme și să poată fi folosit și cu noile tehnologii de comunicație, mai rapide. Soluția acestei probleme a fost elaborarea conceptului de **cablaj structurat**, definit apoi prin mai multe standarde internaționale.

Standardul *EIA-TIA 568A* se referă la cablarea edificiilor comerciale, specificând structura cablajului, configurația minimală necesară, categoriile de cabluri și componente care trebuie folosite, modul de instalare, caracteristicile de performanță care trebuie satisfăcute, limitele acceptabile de distanțe și alți parametri, cât și modalitățile de verificare a acestora. Este abordată deasemenea și problema realizării cablajului la nivel mai complex pentru un grup de clădiri, astfel un proiect complex de cablaj structurat presupune realizarea unei structuri ierarhice, arborescente, cu posibilitatea adăugării ulterioare de legături redundante. Specificațiile standardului referă câteva din următoarele aspecte și anume:

- Cerințe minime pentru realizarea cablării unei clădiri:
 - topologia cablării și distanțele premise;
 - elemente componente ale cablajului;
 - medii de transmisie folosite cu specificarea parametrilor necesari;
 - modul de realizare a cablării verticale, respectiv orizontale;
 - mod de identificare a cablurilor folosite;
 - documentarea proiectului;
- subsisteme componente ale sistemului de cablare structurată:
 - subsistem de la intrarea în clădire;
 - camera cu echipamente;
 - cablarea coloanei vertebrale;
 - dulapul pentru telecomunicații;
 - cablarea orizontală;
 - componentele zonei de lucru

Topologia cablării specificată în standardul *EIA-TIA 568A* este de tip stea, organizată ierarhic (stea extinsă). Centrul topologiei îl constituie comutatorul principal, al doilea nivel ierarhic îl constituie comutatorul intermediar aferent unui edificiu al zonei, iar la nivelul de jos se găsește oficiul pentru telecomunicații asociat unui etaj sau unui grup de încăperi.

Elementele constitutive sunt:

- *comutatorul principal* – centrul de distribuție către celelalte edificii;
- *comutatoarele intermediare* – sunt locale edificiilor;
- *oficiul pentru telecomunicații* – reprezintă dulapurile de distribuție locale pentru cablurile la care se conectează stațiile sau aferente cablării verticale;
- *tronsonul interedificii* – identifică cablurile principale care interconectează centrul de distribuție principal;
- *tronsonul intern* - conectează comutatorul intermediar cu oficiile de distribuție;
- *Încăperea cu echipamente* – corespunzătoare unui plan de cablare cu elemente pasive sau active;
- *Infrastructura de intrare*- pentru interfațarea sistemului de cablare exterior cu cel interior;
- *zona de lucru* – posturile de lucru, cablurile de interconectare, adaptoare externe între cabluri;
- *panouri intermediare* – identifică panourile de conectare pentru mediile de transmisie;
- *blocuri cu terminatori* – reprezintă terminatorii mecanici ai cablurilor;
- *prize pentru comunicație, adaptoare pentru cablaj*.

Mediile de transmisie uzuale sunt:

- cablu torsadat UTP de categorie mai mare decăt 3 sau STP;
- fibra optică multimodală sau monomodală;
- cablu coaxial subție sau gros.

Tipurile de conectori utilizați sunt:

- conectori RJ-45 pentru cablu UTP;
- conectori hermafrodiți pentru cablu STP;
- conectori pentru fibra optică de tip SC sau ST;
- conectori BNC pentru cabu coaxial de distribuție orizontală.

Astfel, pentru a face posibilă o administrare cât mai ușoară și mai eficientă a rețelei, cablajul este structurat folosind concentratoare (pe diverse niveluri). La fiecare etaj trebuie să se implementeze un concentrator de etaj, iar dacă aria care trebuie acoperită este prea mare, atunci se pot utiliza mai multe concentratoare. La posturile de lucru cablul UTP este terminat în cutii de conectare RJ-45, iar la concentrator, în cutii sau panouri de conectare (patch panel). Lungimea cumulată a cablului și a patch cord-urilor UTP folosite pentru legarea unui calculator la echipamentul din concentrator nu are voie să depășească 100m. În concentratorul de etaj, se plasează hub-uri, switch-uri sau alte echipamente.

Cablurile trebuie etichetate corespunzător și aerisirea trebuie să fie suficientă pentru a preveni supraîncălzirea echipamentelor. Concentratoarele de etaj sunt legate la concentratoarele de clădire, legătură care se poate realiza cu cablu de categoria 5 sau cu fibră optică monomod. Se pot proiecta și realiza legături redundante atât între concentratoarele de etaj cât și între cele de clădire. Concentratorul de grup de clădiri este legat de concentratoarele de clădiri cu fibră optică multimod (cel mult 2km) sau monomod (cel mult 3km). Normele de instalare se referă la instalarea cablurilor (tensiune maxim admisă pe fir, mod de conectare mecanic), cablarea orizontală mascată, protecția la împământare, protecția specifică cablurilor cu fibre optice.

3. Desfășurarea lucrării

3.1 Se vor discuta topologiile de rețele de calculatoare punându-se în evidență avantajele și dezavantajele acestora.

3.2 Se vor discuta funcțiile următoarelor dispozitive de rețea: placă de rețea, concentrator, repetor, hub, bridge, switch și router.

3.3 Se vor discuta aspectele legate de cablajul structurat și standardul EIA-TIA 568A.

3.4 Se va analiza cablarea etajului și se vor pune în evidență elementele cablajului structurat.

LUCRAREA NR. 4

METODE DE ACCES LA MEDIU

1. Scopul lucrării

Obiectivul acestei lucrări este cunoașterea metodelor de acces la mediu pentru rețelele ETHERNET, TOKEN-BUS și TOKEN-RING și descrierea restricțiilor impuse în proiectarea rețelelor de calculatoare.

2. Considerații teoretice

2.1 Standardul IEEE 802.3 și ETHERNET

Primul standard Ethernet a fost publicat în anul 1980 de către un consorțiu al Digital Equipment Company, Intel, și Xerox (DIX). În 1985, Institute of Electrical and Electronics Engineers (IEEE) a publicat standardele pentru LAN-uri numind 802.3 standardul Ethernet. Pentru ca standardul să fie compatibil cu modelul International Standards Organization (ISO)/OSI, standardul 802.3 adresează necesitățile Nivelului 1 și a porțiunii inferioare a Nivelului 2 ale modelului OSI. Ca urmare standardul 802.3 este puțin modificat față de standardul Ethernet însă cele două standarde sunt aproape identice. Rețelele Ethernet rulează la diferite viteze: 10Mbps, 100Mbps, 1Gbps și 10Gbps. Toate standardele sunt în esență compatibile cu standardul Ethernet original. Metoda de acces la mediu folosită în rețelele Ethernet este CSMA/CD (Carrier Sense Multiple Access with Collision Detection).

Metoda de acces la mediu CSMA/CD operează, în trei etape:

- **sesizarea purtătoarei** (carrier sense) - stația care dorește să transmită, trebuie să ‘asculte’ întâi magistrala, pentru a determina dacă este ocupată;
- **accesul multiplu** (multiple access) - o stație care a ascultat magistrala și a detectat că este liberă, poate iniția o transmisie, inițierea simultană putând genera coliziune;
- **detectarea coliziunii** (collision detection) - este realizată de către fiecare stație care este în transmisie, deoarece fiecare stație ascultă linia în timpul transmisiei (*listen while talking*).

Prima stație care detectează o coliziune va suspenda transmisia și va transmite în linie un semnal special, care anunță interferența (*jamming*), semnal care va permite ca toate stațiile care sunt conectate la mediu să ia cunoștință despre coliziune, suspendându-și activitatea. Durata de așteptare până la reluarea transmisiei este variabilă, dată de un algoritm de revenire. Domeniul de manifestare al coliziunii este rețeaua locală însăși, în sensul că stațiile separate de huburi sau repetoare vor face parte din domeniul de acțiune al coliziunii, pe când stațiile separate prin switchuri sau bridge-uri, fac parte din domenii de coliziune diferite, pentru că acestea nu propagă semnalul de coliziune.

Pentru corecta funcționare a algoritmului, este necesară asigurarea condiției ca stația transmițătoare să ia cunoștință de o eventuală coliziune, înainte de a termina transmisia. Cazul critic în cazul rețelei Ethernet care rulează la viteza de 10Mbps este constituit de următoarea situație:

- stațiile A și B aflate la extremitățile rețelei sunt separate prin repetitoare;
- cele doua statii sunt separate de 4 repetitoare (maximum posibil admis);
- rețeaua este cablată cu cablu coaxial gros;
- cand cadrul este foarte aproape de stația B, aceasta inițiază și ea o transmisie;
- are loc o coliziune în imediata vecinătate a stației B, semnalul de coliziune se propagă spre stația A;
- stația A detectează coliziunea și oprește transmisia în curs.

Întârzierea de propagare a coliziunii (calculată prin adunarea timpului necesar ca semnalul de date să parcurgă traseul de la stația A la stația B, cu timpul necesar semnalului de coliziune să străbată traseul de la stația B la stația A), are valoarea de 49,9μs. Un pachet cu lungimea minimă de 512 biți (64 octeți), pentru o viteză de transmisie de 10Mbps necesită pentru transmisie un timp de 51,2μs, care este mai mare decât întârzierea de propagare a coliziunii.

Operațiile necesare implementării metodei de acces la mediu sunt:

- transmiterea de pachete, operație care presupune că subnivelul MAC să accepte un pachet de la nivelul superior și să formeze pe baza lui fluxul de biți transmis către mediul fizic;
- receptarea de pachete, operație prin care subnivelul MAC recepționează fluxul de biți de la nivelul fizic și formează pachetul pe care-l înaintează nivelului superior, prin punctele de acces la serviciile corespunzătoare;
- așteptarea pentru inițierea transmisiei, până la eliberarea canalului;
- generarea și interpretarea la recepție, a câmpului de control al erorii FCS;
- spațierea temporală corespunzătoare a cadrelor transmise (*inter-frame spacing*);
- planificarea retransmisiei după detectarea unei coliziuni.
- emiterea unui semnal special de anunțare a detectării coliziunii (*jamming*);
- generarea, la emisia fiecărui cadru, a unui câmp preambul, respectiv eliminarea sa în procesul recepției cadrului.

Cadrul Ethernet are o lungime variabilă, cuprinsă între 64 și 1518 octeți. Valoarea minimă este calculată din considerente de detectare a coliziunii, iar cea maximă din considerente de timp de ocupare a mediului. Figura 5.1 definește formatul unui cadru Ethernet, semnificația câmpurilor fiind descrisă în cele ce urmează.

Octeți	7	1	6	6	2	64-1518	4
	Preambul	SFD	DA	SA	Tip	Info	FCS

Figura 5.1 Structura cadrului Ethernet

Câmpul **Preambul**, cu o lungime de 7 octeți, necesar stațiilor care primesc pachetul pentru a se sincroniza cu ceasul stației transmițătoare. Câmpul **SFD** (*Start Frame Delimiter*), marchează începutul cadrului. Câmpurile **DA** (*Destination Address*) și **SA** (*Source Address*) conțin adresele stației destinație și respectiv sursă. Câmpul **Tip**, conține un cod al protocolului de nivel superior care este generatorul sau destinatarul unității de date conținute în câmpul de informație **Info**. Câteva coduri ale protocoalelor cele mai semnificative sunt date de tabelul următor:

Tabelul 5.1 Codurile protocoalelor

Valoare camp (hexa)	Tip protocol
0000-05DC	Camp Lungime pentru IEEE 802.3
0600	Xerox
0800	DoD IP
0805	X.25 Level 3
0806	ARP

Câmpul **FCS** (*Frame Control Sequence*) reprezintă valoarea sumei de control CRC, calculat pentru câmpurile anterioare.

Se remarcă faptul ca pachetul nu prevede un câmp de sfârșit de cadru, asemănător cu câmpul începutului de cadru **SFD**, acest rol fiind jucat de spațierea între cadre, ce prevede un interval minim temporal între două cadre consecutive de 9,6 μs (valoarea standard pentru parametrul IPG – *Inter Packet Gap*).

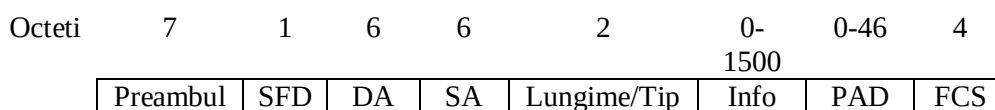


Figura 5.2 Structura cadrului MAC 802.3

Cadrul de date MAC 802.3 prezentat în figura 5.2 are o lungime variabilă, cuprinsă între 64 și 1518 octeți. Semnificația câmpurilor este descrisă în cele ce urmează.

Primul câmp al său este campul **Preambul**, format din 7 octeți, pe baza căruia are loc sincronizarea stației receptoare cu ceasul stației transmiatoare a respectivului cadru.

Câmpul **SFD** (*Start Frame Delimiter*), marchează începutul cadrului. Câmpurile **DA** (*Destination Address*) și **SA** (*Source Address*) conțin adresele stației destinație și respectiv sursă.

În cazul în care câmpul **Lungime/Tip** (*Length/Type*) are o valoare mai mica decat 0x600 acest camp indică lungimea câmpului următor, de date (campul **Info**). În cazul în care câmpul **Lungime/Tip** are o valoare mai mare sau egală cu 0x600 acest câmp indică tipul care reprezintă protocolul de nivel superior (spre exemplu 0x800 reprezintă IPv4 sau 0x806 reprezintă ARP).

De regulă campul **Info** conține pachetul reprezentând unitatea de date a protocolului de nivel superior (LLC-PDU).

Dacă lungimea câmpului de date nu depășește 46 de octeți, este necesară inserarea unui câmp suplimentar, numit **PAD**, continând caractere ‘pad’, care să contribuie prin octeții săi la asigurarea lungimii minime de 64 de octeți pentru un cadru MAC 802.3.

Câmpul **FCS** (*Frame Control Sequence*) conține valoarea sumei de control pe baza polinomului CRC, calculată pentru câmpurile precedente.

Cadrul MAC 802.3 nu conține un câmp marcator (delimitator) al sfârșitului de cadru.

Exista 3 tipuri de rețele Ethernet rulând la viteza de 10Mbps: 10BASE5, 10BASE2 și 10BASE-T. Aceste tipuri împart câteva caracteristici comune: parametri de timp, formatul cadrului, procesul de transmisie și regula de proiectare de bază. Toate rețelele Ethernet de 10Mbps folosesc codificarea Manchester. Codificarea Manchester presupune o tranzitie la

mijlocul bitului a carei direcție determină valoarea binară a bitului. Dacă tranziția este de la 1 la 0 bitul are valoarea 0, iar dacă tranziția este de la 0 la 1 bitul are valoarea 1. Rețelele 10BASE5 folosesc cablu coaxial gros pentru care lungimea maximă permisă fără folosirea repetoarelor este de 500m. Rețelele 10BASE2 folosesc cablu coaxial subțire pentru care lungimea maximă permisă fără folosirea repetoarelor este de 185m. Ambele tipuri de rețele folosesc transmisia half-duplex. Topologia acestor rețele este de tip magistrală așa cum se vede în figura.

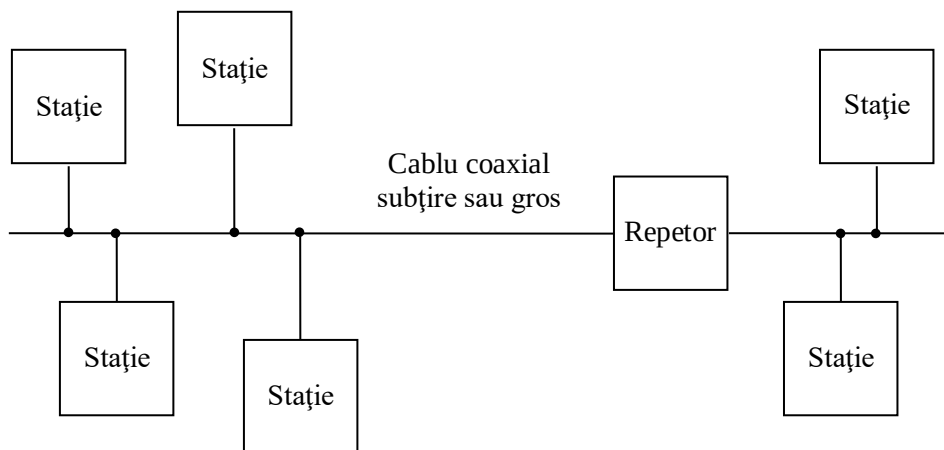


Figura 5.3 Topologia rețelei 10BASE2

Rețelele 10BASE-T folosesc cablu UTP pentru care sunt standardizate următoarele distanțe maxime: 90m pentru cablarea orizontală, 3m patch cord de la calculator la priză și 6m patch cord de la patch panel la hub sau switch. Așadar lungimea maximă permisă este 99m însă se obișnuiește să se rotunjească la 100m. Acest tip de rețea folosește atât transmisia half-duplex cât și cea full-duplex. Topologia acestor rețele este stea sau stea extinsă așa cum se vede în figura.

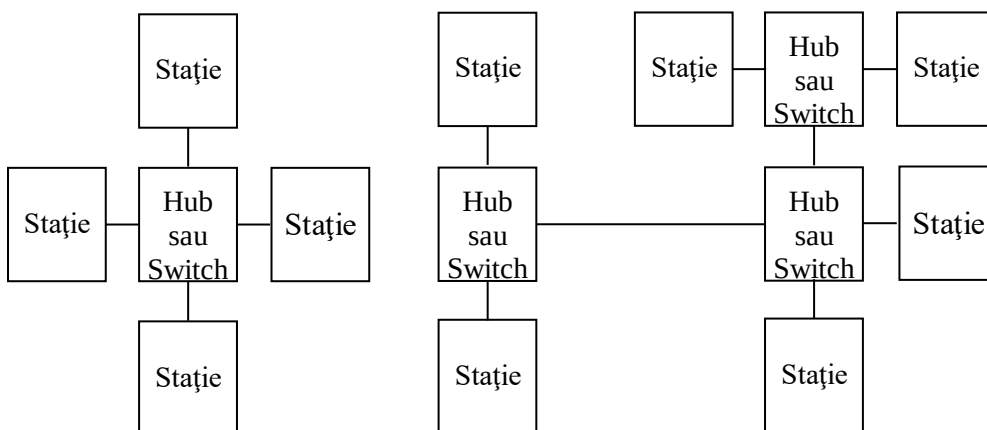


Figura 5.4 Topologia rețelei 10BASE-T

În cazul rețelelor Ethernet de 10Mbps se folosește pentru proiectare regula 5-4-3 care permite maxim 5 segmente separate de maxim 4 repetoare și maxim 3 segmente populate între oricare două stații.

Exista 2 tipuri de rețele Ethernet rulând la viteza de 100Mbps: 100BASE-TX și 100BASE-FX. Aceste tipuri împart câteva caracteristici comune: parametri de timp, formatul cadrului și părți din procesul de transmisie. Rețelele Ethernet de 100Mbps numite și Fast Ethernet folosesc același format de cadru ca și rețelele Ethernet de 10Mbps. Rețelele Fast Ethernet folosesc două codificări. Prima codificare este 4B/5B iar a doua este specifică mediului de transmisie care este cupru sau fibră optică.

Retelele 100BASE-TX folosesc cablu UTP Cat 5. Prima codificare este 4B/5B iar a doua este multi-level-transmit-3 sau MLT-3. Codificarea MLT-3 folosește 3 nivele de tensiune și presupune prezenta sau absenta tranziției la mijlocul bitului indicând valoarea binară 1 respectiv 0. Acest tip de rețea folosește atât transmisia half-duplex cât și cea full-duplex. Topologia acestor rețele este stea sau stea extinsă.

Retelele 100BASE-FX folosesc fibra optică. Prima codificare este 4B/5B iar a doua este NRZI. Codificarea NRZI presupune prezența sau absența tranziției la mijlocul bitului indicând valoarea binară 1 respectiv 0. Acest tip de rețea folosește transmisia full-duplex având două căi separate pentru transmisie și recepție. Cel mai adesea aceste rețele se folosesc pentru cablarea verticală.

În cazul proiectării rețelelor Fast Ethernet se poate folosi maxim 1 repetor de clasa I sau 2 repetitoare de clasa II între oricare două stații. În cazul în care se folosesc două repetitoare de clasa II acestea vor fi conectate printr-o legătură de maxim 5m. Un repetor de clasa I poate introduce o latență de până la 140 timpi de bit iar un repetor de clasa II poate introduce o latență de până la 92 timpi de bit.

Există 3 tipuri de rețele Ethernet rulând la viteza de 1000Mbps: 1000BASE-TX, 1000BASE-SX și 1000BASE-LX. Aceste tipuri prezintă parametri de timp identici. Rețelele Ethernet de 1000Mbps numite și Gigabit Ethernet folosesc același format de cadru ca și rețelele Ethernet de 10 și 100Mbps. Retelele Gigabit Ethernet folosesc două codificări.

Retelele 1000BASE-TX folosesc cablu UTP Cat 5e și transmit full-duplex pe toate 4 perechile de fire torsiadate. Codificarea folosită este 4D-PAM5. Acest tip de rețea folosește atât transmisia half-duplex cât și cea full-duplex. Topologia acestor rețele este stea sau stea extinsă.

Retelele 1000BASE-SX și 1000BASE-LX folosesc fibra optică. Rețelele 1000BASE-SX folosesc emițătoare LED sau laser de 850nm și fibra optică multimod de 62,5 sau 50 micrometri. Retelele 1000BASE-LX folosesc emițătoare laser de 1310nm și fibra optică monomod de 10 micrometri sau multimod de 50 sau 62,5 micrometri. Prima codificare este 8B/10B iar a doua este NRZ. Metoda MAC tratează legătura ca și punct-la-punct. Acest tip de rețea folosește transmisia full-duplex având două căi separate pentru transmisie și recepție. Gigabit Ethernet permite doar un singur repetor între două stații. Topologia acestor rețele este stea sau stea extinsă.

Exista 7 tipuri de rețele Ethernet rulând la viteza de 10Gbps: 10GBASE-SR, 10GBASE-LX4, 10GBASE-LR, 10GBASE-ER, 10GBASE-SW, 10GBASE-LW și 10GBASE-EW. Având formatul cadrului și alte specificații de Nivel 2 Ethernet compatibile cu standardele anterioare, similaritățile dintre rețelele 10GbE și versiunile anterioare sunt remarcabile și permit interoperabilitate între toate standardele Ethernet. Suportul pentru distanțe foarte mari ajungând până la 40km și compatibilitatea cu rețelele SONET și SDH fac din rețelele 10GbE

jetonul. Sunt definite 4 clase de priorități, 0, 2, 4 și 6, prioritatea crescând de la 0 la 6. Fiecare prioritate are propria sa coadă de cadre de transmis. În acest fel se garantează traficului de prioritate 6 o fracțiune cunoscută din lățimea de bandă a rețelei care poate fi folosită pentru implementarea traficului de timp real.

Formatul cadrului 802.4 este prezentat în figura 5.6, semnificația câmpurilor fiind descrisă în cele ce urmează.

Octeti ≥	1	1	1	2 sau 6	2 sau 6	0-8182 sau 0- 8174	4	1
	Preambul	SD	FC	DA	SA	Date	FCS	ED

Figura 5.6 Structura cadrului 802.4

Câmpul **Preambul** este folosit pentru sincronizarea ceasului receptorului. Câmpurile **SD** (*Start Delimiter*) și **ED** (*End Delimiter*) marchează începutul și sfârșitul cadrului și sunt unele dintre simbolurile speciale utilizate pentru controlul rețelei.

Pentru cadrele de date câmpul **FC** (*Frame Control*) conține prioritatea cadrului. În acest caz, câmpul poate conține un indicator care cere stației destinație confirmarea recepției corecte sau incorecte a cadrului. Pentru cadrele de control acest câmp specifică tipul cadrului. Cadrele de control sunt următoarele:

Tabelul 5.2

Valoare	Nume
00000000	CLAIM_TOKEN
00000001	SOLICIT_SUCCESOR_1
00000010	SOLICIT_SUCCESOR_2
00000011	WHO_FOLLOWS
00000100	RESOLVE_CONTENTION
00001000	TOKEN
00001100	SET_SUCCESOR

Câmpurile **DA** (*Destination Address*) și **SA** (*Source Address*) conțin adresele destinației și sursei pe 2 sau 6 octeți la fel ca în rețelele 802.3. Ca și în cazul rețelelor 802.3, o rețea 802.4 poate utiliza numai adrese pe 2 sau 6 octeți.

Câmpul **Date** conține datele transmise și poate avea două lungimi. Când sunt utilizate adrese pe 2 octeți câmpul date poate avea până la 8182 octeți. Când sunt utilizate adrese pe 6 octeți câmpul date poate avea până la 8174 octeți.

Câmpul **FCS** (*Frame Control Sequence*) este folosit pentru detectarea erorilor. Este folosit același algoritm și polinom ca și în cazul rețelelor 802.3.

Interfața păstrează intern adresele stațiilor predecesor și succesori. Periodic deținătorul jetonului trimite unul din cadrele SOLICIT_SUCCESOR pentru a solicita cereri de la stațiile care doresc să intre în inel. Cadrul conține adresa stației transmițătoare și adresa stației succesori. Doar stațiile cuprinse între aceste limite pot cere intrarea în inel. Dacă nici o stație nu cere să intre în inel, stația care deține jetonul își continuă activitatea normală. Dacă o stație dorește să intre, este inserată în inel și devine succesorul deținătorului jetonului. Dacă mai

multe stații doresc să intre, cadrele acestora se vor ciocni și vor fi distruse ca și în cazul rețelor 802.3. Deținătorul jetonului va rula un algoritm de arbitraj de tip număratoare inversă binară care începe cu difuzarea unui cadru RESOLVE_CONTENTION.

Dacă o stație dorește să părăsească inelul, transmite un cadru SET_SUCCESSOR spre stația predecesor care marchează stația succesor.

Inițializarea inelului este un caz special de introducere de noi stații în inel. Prima stație pornită observă că nu există trafic pentru o anumită perioadă de timp și transmite un cadru CLAIM_TOKEN. Deoarece nimeni nu concurează pentru jeton, stația creează un inel care conține doar aceasta stație. Periodic se introduc noi stații în inel pe măsură ce acestea sunt pornite. Dacă primele două stații sunt pornite simultan, inițializarea inelului presupune cererea jetonului de către stații folosind algoritmul de tip numărătoare inversă binară.

Stația care transferă jetonul ascultă să vadă dacă succesoul ei transmite un cadru sau transferă jetonul. Dacă nu se întâmplă nimic jetonul este transferat a doua oară. Dacă nici de această dată nu se întâmplă nimic stația care deține jetonul consideră că stația succesor este căzută și transmite un cadru WHO_FOLLOWS care conține adresa succesoului. Succesorul stației căzute transmite un cadru SET_SUCCESSOR către stația care deține jetonul numindu-se ca nou succesor. Dacă succesorul succesoului stației care deține jetonul este căzut, stația transmite un cadru SOLICIT_SUCCESSOR_2. Inelul este refăcut prin rularea algoritmului de tip numărătoare inversă binară pentru toate stațiile care doresc să intre în inel.

Dacă stația care deține jetonul cade și jetonul se pierde, problema se rezolvă folosind algoritmul de inițializare a inelului deoarece fiecare stație are un ceas care este reinițializat la apariția unui cadru în rețea. Când acest ceas atinge o valoare de prag, stația emite un cadru CLAIM_TOKEN și algoritmul de tip numărătoare inversă binară determină stația care primește jetonul.

În cazul în care în rețea sunt mai multe jetoane, stația sau stațiile deținătoare de jeton care sesizează o transmisie de la o altă stație renunță la propriul jeton. În cazul în care, accidental, sunt eliminate toate jetoanele, lipsa activității va determina una sau mai multe stații să încerce obținerea unui nou jeton.

2.3 Standardul IEEE 802.5 – TOKEN-RING

La începutul anilor '80, organizația IEEE a creat comitetul 802.5, pentru specificarea nivelului fizic și a subnivelului MAC al rețelei Token Ring, standard acceptat și de ISO și publicat ca standard ISO 8802.5 în anul 1992. O rețea Token Ring constă din mai multe stații legate într-o topologie de inel fizic. Cablarea rețelei se face sub forma stelară, centrul stelei îl reprezintă concentratorul sau centrul de cablaj, de la care pleacă legăturile către stațiile din rețea. Mediul de transmisie folosit în rețelele Token-Ring este cablul STP sau UTP. Dacă una din stații se defectează sau trebuie dezactivată temporar, operația de scoatere a stației din rețea se face la nivelul centrului de cablaj, prin acționarea unor relee de ocolire sau trecere bypass după cum se vede în figura 5.7.

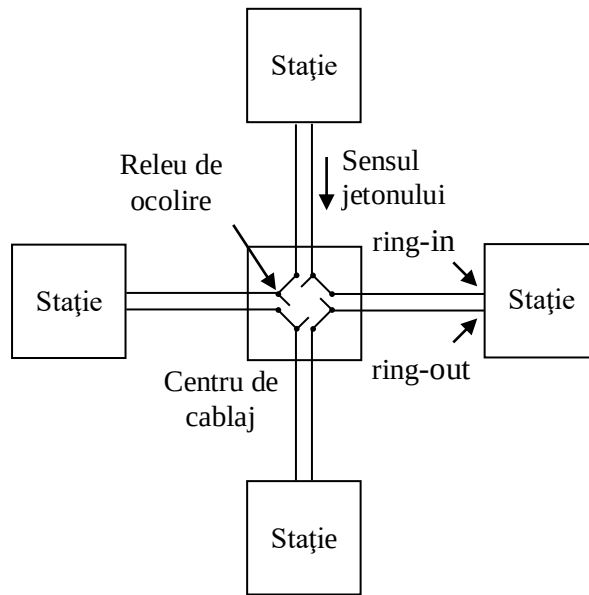


Figura 5.7 Topologia rețelei Token-Ring

Metoda de acces la mediu token ring este o metodă de tip 'token passing' (pasare de jeton) bazată pe existența în rețea a unui pachet special, numit jeton sau *token*. Acest pachet, de lungime minimă, circulă în rețea indicând că mediul este liber. O stație poate transmite doar când achiziționează tokenul. La terminarea transmisiei, sau după un timp determinat, ea este obligată să elibereze tokenul. La inițializarea rețelei, stația care este desemnată să gestioneze rețeaua, monitorul activ, va genera tokenul.

Implementarea metodei de acces, așa cum o face standardul 802.5, are la bază următoarele specificații:

- protocolul MAC este bazat pe existența în rețea doar a unui singur token, astfel că o stație care a terminat transmisia nu va mai genera alt token, ci va elibera tokenul deținut;
- există biți pentru prioritate care pot fi setați la nivelul fiecărei stații;
- există un bit monitor, folosit dacă este utilizată gestionarea centralizată a inelului;
- există indicatori de rezervare, folosiți de stațiile cu prioritate ridicată pentru a specifica că următorul token va fi cu prioritate crescută;
- există timere pentru controlul perioadei de păstrare a tokenului de către o stație pentru a nu ocupa abuziv inelul;
- există biți de achitare și biți pentru semnalarea unor erori sau a îndeplinirii unor acțiuni.

Implementarea metodei de acces la mediu se realizează la subnivelul MAC. Formatul cadrului 802.5 este prezentat în figura 5.8 semnificația câmpurilor fiind descrisă în cele ce urmează.

Octeti	1	1	1	6	6	0-30	4-17749	4	1	1
	SD	AC	FC	DA	SA	RI	Info	FCS	ED	FS

Figura 5.8 Structura cadrului 802.5

Câmpul **SD** (*Starting Delimiter*) are rolul de identificare al începutului fizic al cadrului. Câmpul **AC** (*Access Control*) conține informații pentru accesul la mediu (la inel). Structura sa de bit este PPPTMRRR, unde subcâmpurile au următoarea semnificație:

- subcâmpul de trei biți PPP indică prioritatea accesului actual;
- bitul T este bitul de token, având valoarea 1, dacă pachetul curent este de date și valoarea 0 dacă este token;
- bitul M este setat în cazul stației monitor;
- subcâmpul de trei biți RRR codifică prioritatea cerută de stație pentru accesul următor.

Cele două câmpuri formează așa numita secvență de start a cadrului **SFS** (*Start-of-Frame Sequence*).

Câmpul **FC** (*Frame Control*) definește conținutul cadrului MAC. Câmpul are structura de bit FFZZZZZZ, unde biții FF identifică natura cadrului:

- cadru MAC de gestionare (control) pentru valoarea 00; este un cadru necesar a fi recepționat de către toate stațiile;
- cadru ce conține în câmpul **Info** un pachet LLC, pentru valoarea câmpului 01.

Biții ZZZZZZ sunt biți de control, cadrele de control folosite de protocolul Token Ring fiind sumarizate de tabelul următor:

Tabelul 5.3

Cod cadru	Nume cadru
00000000	Duplicate Address Test
00000010	Beacon
00000011	Claim token
00000100	Purge
00000101	Active Monitor Presence
00000110	Standby Monitor Presence

Câmpurile **DA** (*Destination Address*) și **SA** (*Source Address*) reprezintă adresele pe 6 octeți ale stației destinate, respectiv ale stației sursă a cadrului curent. Câmpul **RI** (*Routing Information*) conține informația de dirijare necesară dacă cadrul parcurge o rețea extinsă. Câmpul **Info** reprezintă câmpul de date propriu-zis având o lungime de la 4 la 17.749 octeți. Câmpul **FCS** (*Frame Check Sequence*) reprezintă suma de control realizată prin calculul CRC asupra câmpurilor pachetului. Câmpul **ED** (*Ending Delimiter*) indică sfârșitul pachetului și are în structură biți de date și non-data, respectiv are structura de bit: JK1JK1IE. Suplimentar față de alte protocoale, aici se folosesc biții:

- I, pentru indicarea faptului că pachetul curent este unul intermediar;
- E, care semnalizează detectarea unei erori.

Câmpul **FS** (*Frame Status*) este folosit pentru înștiințarea stației sursă despre modul în care a decurs transmisia cadrului. Acest câmp are structura de bit ACrrACrr, unde câmpul rr este câmp rezervat, iar biții A (adresa destinație recunoscută - *Address Recognized*) și C (copiere cadru la destinație - *Frame Copied*) formează combinații cu următoarea semnificație:

Tabelul 5.4

AC	Semnificație
00	stație inexistentă sau inactivă
10	stație existentă, pachet necopiat
11	stație existentă și pachet copiat

Aceste ultime două câmpuri formează secvența de terminare a cadrului **EFS** (*End-of-Frame Sequence*).

Fiecare stație legată la rețeaua Token Ring trebuie să posede un element de cuplare, numit interfață sau unitate de cuplare la mediu. Partea de interfață preia bit cu bit informația din inel transmisă spre stație pe linia de recepție (ring-in). Această informație poate fi transmisă imediat înapoi în rețea, prin punerea ei pe linia ring-out folosind o cale de repetare (*repeat path*), deci în aceste condiții interfața introduce în inel o întârziere de o perioadă de bit.

Din punct de vedere fizic, transceiverul, numit unitatea de cuplare la linie (*Trunk Coupling Unit*), se găsește situat în concentrator, elementul central al topologiei stelare și conține în plus releul pentru decuplarea stației și electronica necesară cuplării la mediu. De asemenea, pentru a se asigura timingul necesar bunei circulații și interpretării a cadrului de token în rețea, este necesar ca întârzierea prevăzută de rețea să fie superioară lungimii tokenului. Dacă această întârziere nu poate fi asigurată prin interfețele existente, se folosesc întârzieri suplimentare, în circuitele de interfață 802.5 ale stațiilor.

Algoritmul de transmisie, repetare și recepție a cadrelor este exemplificat pentru o rețea cu patru stații A, B, C și D presupunând ca stația A dorește să transmită un mesaj către stația C. Transmisia va avea loc după următorul algoritm:

- stația A așteaptă recepția unui token pe linia ring-in pe care, la sesizare, îl reține. Retinerea tokenului semnifică faptul că stația A va seta bitul T din câmpul **AC** al cadrului de token. Astfel stația transformă cadrul de token în cadru de date, iar biții transmiși pe linia ring-out înapoi în rețea, prin intermediul căii de repetare formează secvență de start **SFS**, a cadrului de date;
- după transmiterea secvenței **SFS**, stația A inhibă linia de repetare de bit (*repeat path*);
- stația A începe transmisia propriilor date, prin punerea pe linia ring-out a biților corespunzători câmpurilor de control cadru **FC**, adreselor destinație **DA** și sursă **SA** și eventual informație pentru elementul de interconectare în câmpul **RI**;
- stația A transferă în rețea biții de date din câmpul **Info**;
- dacă stația mai are de transmis date și timpul alocat pentru reținerea tokenului **THT** (*Timer Holding Token*) nu a expirat, setează bitul I (*Intermediate*) din câmpul **ED**, pentru a semnaliza că acest cadru nu este ultimul, apoi continuă transmisia pachetelor succesive, respectând pașii anteriori;
- când stația A a transmis ultimul pachet, resetează bitul I;
- dacă stația a terminat transmisia datelor proprii înaintea receptării înapoi pe linia ring-in a primului cadru transmis, este nevoită să aștepte acest cadru și va transmite în rețea biți de umplere (*fill bit*);
- când stația A recepționează un cadru transmis (îl recunoaște ca propriu prin intermediul câmpului de adresă **SA**), îl scoate din rețea și devine disponibilă să elibereze tokenul. Dacă nu mai are cadre de transmis îl eliberează imediat, dacă mai are date, continuă transmisia în condițiile descrise mai sus;

- în timpul cât stația A a transmis date în rețea, stațiile care nu au posedat jetonul (stațiile B, C și D) au îndeplinit doar funcția de repetare a biților de pe liniile ring-in pe cele ring-out corespunzătoare, dacă ele au detectat eroare de date, semnalează aceasta prin setarea bitului E din câmpul **ED**;
- stația receptoare a mesajului emis de stația A, respectiv stația C, din analiza conținutului câmpului adresă destinație **DA**, recunoaște propria adresa MAC și nu efectuează doar repetarea biților, ci copierea lor pe linia 'recepție', în bufferele proprii și deasemenea setează corespunzător biții A și C din câmpul **FS** al cadrului recepționat;
- stația A, la sfârșitul recepției înapoi a cadrului (cadrelor) transmise, va elibera tokenul și va reactiva linia de repetare proprie (*repeat path*).

De remarcat că, în anumite cazuri, depinzând de viteza de transmisie și de numărul de stații conectate la rețea, pentru a se îmbunătăți performanța rețelei, se poate aplica regula eliberării anticipate a tokenului (*early token release*). Prin aceasta, stația care a transmis un cadru (sau o secvență) nu mai este obligată să aștepte până la întoarcerea cadrului transmis pentru a elibera tokenul, ci poate să o facă anticipat.

Cadrelor MAC 802.5 sunt transmise unul după altul în secvență, fără a exista pauza între două pachete consecutive, precum se impunea la secvența de cadre 802.3. Secvența de transmisie fiind continuă, sincronizarea între stații se menține permanent. În situația căderii unei stații din inel, este posibil să se piardă sincronizarea între stații, motiv pentru care standardul prevede posibilitatea ca primul token și primul cadru dintr-o secvență să fie precedați de un număr de biți de sincronizare.

Funcțiile de control și gestionare a rețelei se pot grupa în patru clase, care se referă la:

- stabilirea monitorului activ (*active monitor*), ce reprezintă stația din inel care generează tokenul, stabilește ceasul de referință în inel, generează procesul periodic de notificare a stațiilor vecine sau recuperează tokenul pierdut;
- stabilirea grupului de parametri valabili pentru toate stațiile din inel (*ring parameter server*);
- monitorizarea erorilor din inel și elaborarea de statistici (*ring error monitor*).
- elaborarea unui raport de configurare (*configuration report server*), funcție ce colectează de la stații informații de configurare.

Toate aceste funcții sunt realizate prin intermediul unor adrese funcționale (*functional addresses*), fiind adrese de multicast administrate local.

Regula de stabilire a stației cu rol de monitor activ statuează că la un moment dat în rețea există un singur monitor activ (*active monitor*), stabilit printr-un proces de alegere, celelalte stații fiind într-o stare de așteptare (*standby monitor*).

În faza de alegere a monitorului activ, toate stațiile candidate transmit continuu un cadru special 'Claim token'. Acest pachet conține, pentru fiecare stație, propria valoare de solicitare pentru a deveni monitor activ, valoare stabilită pe baza adresei sale în inel. În timpul procesului de alegere, fiecare stație compară valoarea de solicitare (*claim value*) din pachetul recepțat, cu propria valoare. Dacă valoarea proprie este superioară celei recepționate continuă emiterea de pachete de solicitare, iar dacă valoarea proprie este inferioară valorii din pachetul recepționat, întrerupe emisia de pachete 'Claim token' și doar repetă la ieșire pachetele

recepționate. În acest fel, la sfârșit doar o singură stație va recepționa doar propria valoare de *claim*, devenind monitorul activ.

Monitorul activ își va face simțită prezența în rețea prin generarea periodică a unui pachet AMP (*Active Monitor Presence*) către toate stațiile din inel. Dacă o stație din inel de tip monitor în așteptare (*standby monitor*) nu recepționează un astfel de pachet o perioadă mai lungă decât durată TSM (*Timer Standby Monitor*), poate iniția un nou proces de alegere.

Cadrelor de control AMP, generate cu o perioadă de repetiție dată de valoarea parametrului TAM (*Timer Active Monitor*) mai au încă un rol, și anume de notificare a stațiilor vecine (*neighbor notification*). Prin acest proces, fiecare stație din inel va notifica adresa stației active din amonte, cea legată la linia ring-in a stației curente. Adresa este notată UNA (*Upstream Neighbor Address*), sau NAUN (*Nearest Active Upstream Neighbor*). Procesul de notificare se desfășoară în următoarea secvență:

- prima stație activă ce urmează monitorului activ, la primirea unui cadru AMP, execută următoarele operații: setează pe 1 biții A (*Address Recognized*) și C (*Frame Copied*), din câmpul de stare cadru FS, copiază cadrul AMP recepționat și memorează adresa stației emițătoare (a stației vecine) într-o locație de memorie notată SUA (*Stored Upstream neighbor's Address*), transmite către toate stațiile un cadru SMP (*Standby Monitor Presence*);
- prima stație activă în aval de stația transmițătoare a cadrului SMP, va recepționa acest cadru, va memora adresa stației sursă a cadrului și va genera și ea un cadru SMP, pentru a notifica prezența sa.

Accesul bazat pe priorități este o soluție pentru posibilitatea folosirii rețelei în aplicații diverse, fapt ce necesită tratarea diferențiată a acțiunilor (cadrelor), după prioritatea lor. Spre exemplu, la o rețea locală pot fi conectate nu numai calculatoare, ci și elemente de supraveghere sau de conducere a proceselor. Acestea, lucrând în timp real, vor necesita tratarea lor cu prioritate față de pachetele obișnuite de date schimbate de două calculatoare. Pentru aceasta, rețeaua 802.5 prevede un mecanism de acces la mediu cu priorități, realizat prin intermediul perechilor de subcâmpuri de biți de prioritate și de rezervare (subcâmpurile PPP și RRR din câmpul de control al accesului AC), din cadrul unui pachet de date, sau din cadrul tokenului.

Regula de bază a accesului este că o stație nu poate transmite cadre în rețea dacă nu posedă o prioritate mai mare, sau cel puțin egală, cu cea a tokenului care circulă în rețea și pe care are dreptul să-l rețină.

Câteva reguli de acces pe bază de priorități sunt:

- o stație care dorește să transmită, având o anumită prioritate, trebuie să solicite generarea unui token cu acea prioritate prin setarea la valoarea priorității sale a biților de rezervare dintr-un pachet în transit;
- stația care generează un token, sau eliberează tokenul după terminarea unei transmisii, poate ridica prioritatea tokenului, funcție de valoarea înscrisă în câmpul de rezervare; tokenul inițial eliberat de monitorul activ are prioritatea 0;
- un pachet de date, pe timpul traversării inelului, poate avea de mai multe ori modificat subcâmpul de rezervare;
- numai stația care a ridicat valoarea priorității tokenului are dreptul de a o scădea și o va face la următoarea sesizare a tokenului.

Nivelul fizic specificat de standard se referă la aspectele electrice, mecanice, funcționale, asociate cu transmiterea fluxului de biți în mediu. Doar monitorul activ este cel care generează ceasul folosit în rețea. Pentru sincronizarea stațiilor cu ceasul monitorului se prevăd biții suplimentari de sincronizare, motiv pentru care standardul limitează numărul de stații din rețea precum și lungimea maximă a cablurilor folosite.

3. Desfășurarea lucrării

3.1 Standardul IEEE 802.3 și ETHERNET

3.1.1 Se vor discuta aspectele legate de metoda de access la mediu folosită în rețelele Ethernet, tipurile de rețele Ethernet și restricțiile impuse în proiectarea acestor rețele.

3.1.2 Se consideră o rețea CSMA/CD de 10Mbps cablată cu UTP pentru care numărul maxim de repetitoare între oricare două calculatoare aflate în același domeniu de coliziune este 3. Știind că viteza de propagare a semnalului prin cablu este de $200\text{m}/\mu\text{s}$ și timpul de întârziere introdus de un repetor este de $1\mu\text{s}$ să se calculeze lungimea minimă a unui cadru care circulă în această rețea. Lungimea unui cadru este întotdeauna multiplu de octet.

3.1.3 O rețea CSMA/CD de 10Mbps, având lungimea de 50m, are o viteză de propagare de $200\text{m}/\mu\text{s}$. Cadrele de date au o lungime de 1024 biți. Primul interval de bit după o transmisie efectuată cu succes este rezervat receptorului spre a ocupa canalul pentru a transmite un cadru de confirmare. Care este viteza efectivă de date presupunând ca nu sunt coliziuni?

3.2 Standardul IEEE 802.4 – TOKEN-BUS

3.2.1 Se vor discuta aspectele legate de metoda de access la mediu folosită în rețelele Token-Bus.

3.2.2 Parametri unei rețele Token-Bus având 20 de stații și rulând la 10Mbps dau traficului de prioritate 6 un procent de 40% din lățimea de bandă. Care este lățimea de bandă garantată fiecărei stații pentru traficul de prioritate 6?

3.3 Standardul IEEE 802.5 – TOKEN-RING

3.3.1 Se vor discuta aspectele legate de metoda de access la mediu folosită în rețelele Token-Ring și restricțiile impuse în proiectarea și funcționarea acestor rețele.

3.3.2 O rețea Token-Ring de 16Mbps are un ceas pentru păstrarea jetonului cu valoarea de 5ms. Care este cel mai lung cadru care poate fi transmis în acest inel?

3.3.3 O rețea Token-Ring de 4Mbps, având lungimea de 500m are viteza de propagare de $200\text{m}/\mu\text{s}$. Cinci stații sunt uniform răspândite în jurul inelului. Este necesară introducerea de întârzieri suplimentare în inel?

Notițe

LUCRAREA NR. 5

CONECTAREA LA REȚEA

1. Scopul lucrării

Obiectivele acestei lucrări sunt familiarizarea cu componentele de conectivitate, Înțelegerea modurilor de adresare IP, configurarea statică a adresei, configurarea TCP/IP pentru alocare dinamică de adrese, verificarea și testarea conexiunilor TCP/IP, familiarizarea cu conceptele de domeniu și workgrup.

2. Considerații teoretice

2.1 Conectivitatea

O rețea e compusă dintr-o serie de echipamente care comunică între ele. Regulile și procedurile de comunicare care trebuie respectate pentru a realiza comunicarea în cadrul rețelei se numesc protocoale.

Calculatoarele utilizează patru componente pentru a oferi conectivitatea la rețea:

- protocoale
- servicii de rețea
- plăci de rețea
- bindings

Un protocol e un set de reguli și proceduri de comunicare ce definesc conținutul, formatul datelor transmise, modalitățile de sincronizare, secvențiere și control al erorilor.

Serviciile de rețea permit echipamentelor ce utilizează aceleași protocoale să se conecteze și să utilizeze partajat resursele din rețea. Serviciile de rețea oferă funcții de rețea aplicațiilor cum ar fi accesul la fișiere partajate sau imprimante. Aceste servicii sînt componente software înglobate în sistemul de operare sau sînt oferite de terți.

Placa de rețea este componenta hardware care realizează legătura între calculator și mediul de transmisie (cabluri de rețea sau alte medii). Placa de rețea reprezintă interfața fizică între calculator și rețea.

Bindings-urile definesc metodele de legare a componentelor din rețea. Pentru a realiza conectarea la rețea trebuie să se instaleze și configureze protocoalele de rețea, apoi să se instaleze și să se configureze serviciile și în final să se configureze binding-urile dacă e necesar.

2.2 Instalarea TCP/IP pentru conectarea la rețea

Sistemul de operare Linux oferă o implementare completă a tuturor protocoalelor din stiva TCP/IP. Sistemul de operare Windows 2000 instalează stiva TCP/IP ca și stivă implicită de

protocoale (dacă se detectează prezența unei plăci de rețea). Dacă stiva TCP/IP nu a fost instalată puteți să o instalați manual.

Pentru a instala stiva TCP/IP pe sistemul de operare Windows 2000 trebuie să executați următorii pași:

1. Selectați My Network Places și aplicați click dreapta, alegeți Properties.
2. În fereastra Network and Dial-up Connections selectați Local Area Network, aplicați click dreapta și alegeți Properties.
3. Verificați dacă TCP/IP este în lista componentelor instalate. Dacă în lista componentelor afișate sub textul “Components checked are used by this connection”, TCP/IP e bifat atunci apăsați butonul Cancel pentru închiderea ferestrei.
4. Dacă componenta TCP/IP nu e bifată, bifați și apăsați butonul Install. În fereastra Select Network Component Type selectați Protocol și apoi selectați Add.
5. În fereastra Select Network Protocol bifați Internet Protocol (TCP/IP) și apăsați butonul OK.
6. Windows 2000 va instala stiva de comunicație TCP/IP. După instalare se va reveni în fereastra Local Area Connection Properties iar în listă va apărea elementul Internet Protocol bifat. Apăsați butonul OK.
7. Verificați ca Internet Protocol (TCP/IP) să fie bifat și apoi selectați OK.

2.3 Moduri de adresare IP

Adresele IP pot fi alocate manual, dinamic de către serverele DHCP sau automat prin mecanismul APIPA (Automatic Private IP Addressing) – numai în sistemele de operare Windows.

2.3.1 Adresarea statică

Ocazional, în rețea, adresele trebuie asignate manual. De exemplu, configurarea statică a adreselor e necesară într-o rețea cu multe segmente în care nu este prezent nici un server DHCP.

Configurarea statică e recomandată numai în cazul în care nu este disponibil un serviciu DHCP în rețea. Administrarea manuală a adreselor IP este consumatoare de timp în rețelele medii și mari.

2.3.2 Adresarea dinamică

Un server DHCP alocă automat adrese IP dintr-un grup sau grupuri de adrese disponibile, deasemenea, alocă și alte informații TCP/IP pentru configurare cum ar fi adresa serverului DNS, gateway-ul implicit.

Procesul de alocare automată a adresei prin serverul DHCP se desfășoară în următorii pași:

- clientul (echipament din rețea) trimite o cerere pentru primirea unei adrese IP serverului DHCP;
- serverul DHCP deservește echipamentul prin alocarea unei adrese.

2.3.3 Automatic Private IP Addressing – sisteme Windows

APIPA permite configurarea unei adrese IP în cazul în care calculatorul client nu poate primi o adresă IP de la un server DHCP. Prin mecanismul APIPA se alocă doar adresă IP și mască de subrețea, nici o altă informație de configurare. Acest aspect limitează conectivitatea doar la rețeaua locală, calculatorul client neavând acces la alte rețele sau la Internet.

În momentul în care un calculator fără adresă IP pornește, se execută următorii pași:

- calculatorul client încearcă să localizeze un server DHCP pentru a obține parametri de configurare TCP/IP;
- dacă nu se localizează nici un server DHCP, calculatorul client își configurează automat o adresă IP și o mască de subrețea din clasa de adrese rezervate Microsoft (adrese de clasă B din intervalul 169.254.0.1 – 169.254.255.254 și cu masca de subrețea 255.255.255.0);
- dacă, la un moment dat, calculatorul client detectează prezența unui server DHCP, procesul de asignare APIPA este oprit.

2.4 Configurarea TCP/IP pentru asignarea statică a unei adrese IP

Pentru fiecare placă de rețea din calculator care utilizează stiva TCP/IP în momentul în care se configurează static o adresă IP trebuie să se asigneze și masca de subrețea și gateway-ul implicit.

2.4.1 Configurarea TCP/IP pentru asignarea statică a unei adrese IP în sistemele Windows

1. Deschideți fereastra Properties din Local Area Connection.
2. Apăsați Internet Protocol (TCP/IP), verificați dacă e bifată opțiunea TCP/IP și alegeți Properties.
3. Alegeți Use the following IP address.
4. În casuța IP address, scrieți valoarea adresei indicată de instructor.
5. Verificați că opțiunea Use the following DNS server address să fie selectată și în casuța Preferred DNS Server box scrieți valoarea indicată de instructor.
6. Apăsați OK pentru a închide fereastra TCP/IP Properties, apăsați OK pentru a închide fereastra Local Area Connection Properties.

2.4.2 Configurarea TCP/IP pentru asignarea statică a unei adrese IP în sistemele Linux

În sistemele Linux, există două modalități de configurare statică a unei adrese IP, utilizarea command line și configurarea fișierului de rețea. Pentru a realiza aceste configurări trebuie să aveți drepturi de root.

1. Configurarea utilizând command line.

- a) Încărcăți driverul pentru placa de rețea. Unele distribuții vor instala automat driverul pentru placa de rețea, pentru a verifica încărcarea driverului editați fișierul `/etc/modules.conf` sau executați comanda `/sbin/lsmmod`

- b) Pentru a asigura adresa IP și masca de subrețea executați comanda
:/sbin/ifconfig - a interface_name <IPaddress> netmask
<netmaskaddress>
- c) Adăugați gateway-ul implicit executând comanda: /sbin/route add
default gw <gateway_addr>.

Configurarea realizată rămâne valabilă pînă la următoarea repornire a calculatorului.

2. Configurarea utilizînd fișierul de rețea

Fiecare distribuție de Linux își păstrează fișierul de configurare în locuri diferite. Sistemul păstrează configurația pînă cînd se modifică fișierul de configurare. Modificările se păstrează după reboot.

- a) Editați fișierul /etc/sysconfig/network-scripts/ifcfg-interface_name. Fișierul *ifcfg-interface_name* configurat pentru obținerea statică a informațiilor de rețea conține următoarele informații:

DEVICE=interface_name	numele interfeței
USERCTL = no	poate orice utilizator să activeze interfața?
BOOTPROTO=none	cum se alocă adresa IP și ceilalți parametri de rețea (prin DHCP, BOOTP, none - asignarea statică)
ONBOOT=yes	vreți ca interfața să fie activată la reboot?(yes/no)
IPADDR= addrIP	adresa IP alocată interfeței
NETMASK=address	masca de rețea asociată
NETWORK=address	adresa rețelei
BROADCAST=address	adresa de broadcast a rețelei

- b) Editați fișierul /etc/sysconfig/network și completați-l ca în exemplul de mai jos:

NETWORKING = yes	Configurare rețea
HOSTNAME= hostname	numele serverului
FORWARD_IPV4 = yes	realizare IP forwarding
GATEWAYDEV = gwdev	Gwdev este numele device-ului eth# pe care-l folosiți ca să accesați gateway-ului
GATEWAY = gwIP	gwIP este adresa gateway-ului rețelei remote

2.5 Verificarea și testarea conexiunilor TCP/IP

După configurarea stivei TCP/IP, utilizați comenzile `ipconfig/ifconfig` și `ping` pentru a testa configurarea calculatorului și pentru a vă asigura că acesta poate comunica utilizând protocoalele stivei.

2.5.1 Comanda `Ipconfig/Ifconfig`

Ipconfig – comandă utilizată în sistemele Windows pentru a obține informații despre configurarea de bază: adresa IP, masca de rețea, gateway implicit. Prin utilizarea în conjuncție cu `/all` se obține un raport mai detaliat al configurării pentru toate interfețele.

```
Ipconfig [/all] [/renew[adapter]] [/release[adapter]]
```

`/all` = informații despre toate interfețele

`/renew` = reînnoiește informațiile primite de la serverul DHCP pentru toate plăcile de rețea

`/release` = renunță la informațiile primite de la serverul DHCP

Ifconfig – comandă utilizată de sistemele Linux pentru a face o interfață accesibilă nivelului rețea. Aceasta implică asignarea unei adrese IP și a altor parametri și activarea interfeței.

```
Ifconfig [interface] [address [parameters]]
```

Utilizată fără nici un argument, comanda `ifconfig` afișează starea interfețelor active. Dacă se furnizează ca argument o singură interfață atunci se afișează informațiile de configurare doar pentru acea interfață. Dacă se furnizează ca argument `-a` atunci se afișează informațiile despre toate interfețele inclusiv pentru interfețele care nu sînt active. Altfel, se configurează interfața.

`interface` – numele interfeței, de obicei numele urmat de un număr (de exemplu, `eth0` este numele primei interfețe Ethernet);

`up` – argument care activează o interfață;

`down` – argument care dezactivează interfața. Utilizarea acestei opțiuni va duce la ștergerea tuturor intrărilor ce vizează interfața din tabela de rutare.

Pentru o interfață de rețea se pot asigna mai multe adrese IP prin crearea subinterfețelor numite alias. Trebuie să specificați numele_interfeței: număr.

Exemple:

```
ifconfig eth0 192.168.1.1 netmask 255.255.255.0 up
```

```
ifconfig eth0:1 192.168.1.2 netmask 255.255.255.0 up
```

2.5.2 Comanda `ping`. Troubleshooting utilizînd comanda `Ping`

Programul `Ping` (Packet Internet Gopher) e utilizat pentru verificarea accesibilității unui host din rețea. Acest program transmite mesaje ICMP spre un host remote și așteaptă de la acesta un răspuns. Comanda permite determinarea distanței la hostul remote în termeni de timp de transmisie.

```
Ping [-t] [-a] [-n count] [-l length] [-f] [-i ttl] [-r count]
destination_address
```

- t = ping pînă la întrerupe;
 - a = rezolvă adresa host în nume de host;
 - n = numără pachetele de tip echo transmise;
 - l = dimensiunea pachetului echo transmis;
 - f = flag care setează opțiunea de nefragmentare;
 - i TTL = setează timpul de viață pentru un pachet (numărul maxim de hop-uri pe care le poate parcurge pachetul înainte de a fi rejectat din rețea, 1...255);
 - r = afișează ruta parcursă de mesaj;
- destination_address = adresa hostului remote .

Cînd verificați și testați conectivitatea utilizați comanda ping pentru a parcurge următoarea secvență de task-uri:

1. Executați ping pe adresa de loopback pentru a verifica dacă stiva e instalată și configurată corect pe calculatorul local. Pentru aceasta, executați comanda `ping 127.0.0.1`. Dacă comanda eșuează înseamnă că stiva nu răspunde; driver-ul e defect, placa de rețea nu funcționează.
2. Executați ping pe adresa locală pentru a verifica dacă adresa a fost asignată corect. Pentru aceasta executați `ping <IP address of the local host>`.
3. Executați ping pe adresa gateway-ului implicit. Acest pas verifică accesibilitatea gateway-ului implicit și faptul că hostul poate comunica cu alt host din rețea. Pentru aceasta executați `ping <IP address of default gateway>`.
4. Executați ping pe adresa unui host remote localizat dincolo de gateway-ul implicit. Acest pas verifică posibilitatea comunicării cu host-uri din afara segmentului dumneavoastră de rețea.

2.5.3 Comanda tracert/ traceroute

Programul tracert/traceroute reprezintă o unealtă de testare, depanare și management care permite vizualizarea traseului datagramelor IP de la un ruter la altul. Programul funcționează trimițînd pachete ICMP unei adrese IP și incrementînd cîmpul TTL din header-ul IP, începînd cu 1, și analizînd mesajele ICMP returnate. Tracert/traceroute returnează lista ruterelor care au generat mesaje de eroare.

```
Tracert [-d] [-h max_hops] [-j host_list] [-w timeout] target_name
```

- d = implicit, se realizează rezolvarea adreselor în nume, opțiunea activă determină inhibarea acestei translatări;
 - h = numărul maxim de hop-uri în căutare pentru destinația specificată;
 - j = opțiunea de rutare de la sursă;
 - w = specifică un interval de time-out pentru fiecare mesaj de raspuns (ms);
- target_name = numele/adresa hostului remote.

2.6 Workgroup-uri și domenii in Windows 2000

2.6.1 Workgroup-uri

Un workgroup e o grupare logică de calculatoare care împart resurse, fișiere și imprimante. Un workgroup se mai numește și rețea peer-to-peer deoarece toate componentele lui împart resurse în mode egal, fără a exista un server dedicat. Fiecare sistem cu Windows 2000 Server sau Professional din workgroup menține local o bază de date ce conține conturile utilizatorilor și informații legate de securitatea resurselor pentru acel sistem.

Deoarece fiecare calculator din workgroup menține o astfel de bază de date administrarea conturilor utilizatorilor și a resurselor e descentralizată. Un utilizator trebuie să aibă cont de utilizator pe fiecare calculator din workgroup pe care dorește să-l utilizeze. Orice modificare legată de contul utilizatorului (modificarea parolei, crearea unui nou cont) trebuie realizată local, pe fiecare sistem.

Utilizarea workgroup-urilor are următoarele avantaje:

- ușurința în proiectare și implementare;
- nu necesită existența unui calculator pe care să ruleze Windows 2000 Server pentru centralizarea informațiilor legate de securitate;
- este o soluție convenabilă pentru rețele cu puține calculatoare.

2.6.2 Domenii

Un domeniu e o grupare logică de calculatoare care partajează o bază de date centrală în care sînt stocate informațiile despre conturile utilizatorilor și informațiile legate de securitatea domeniului. În sistemele Windows 2000 această bază este cunoscută sub numele de directory și este parte a serviciului Active Directory. Într-un domeniu, directory-ul este localizat pe calculatoarele configurate cu funcția de controlare de domeniu. Un controler de domeniu e un server care realizează administrarea centralizată a domeniului.

Un domeniu nu se referă la o singură locație sau la o anumită configurație de rețea. Calculatoarele dintr-un domeniu pot fi localizate în aceeași proximitate fizică sau pot fi localizate în diferit colțuri ale lumii, comunicînd prin ISDN, DSL etc.

Utilizarea domeniilor are următoarele avantaje:

- administrarea centralizată;
- o singură logare pentru ca utilizatorii să obțină acces la resursele pentru care au permisiuni; un utilizator se poate loga pe un calculator și poate accesa resurse de pe alte calculatoare dacă are permisiuni;
- scalabilitate.

3. Desfășurarea lucrării

3.1 Să se instaleze stiva TCP/IP pentru conectivitate.

3.2 Să se configureze stiva TCP/IP pentru obținerea dinamică a unei adrese IP.

3.2.1 Sistemele Windows.

1. În Control Pane, executați dublu-click pe icon-ul Network and Dial-Up Connections.

2. În fereastra deschisă, selectați Local Area Connection, executați click-dreapta și alegeți Properties.
3. Verificați în lista dacă Internet Protocol (TCP/IP) e bifat și apăsați butonul Properties.
4. În noua fereastră deschisă, bifați opțiunea Obtain an IP address automatically.
5. Apăsați butonul OK.
6. Deschideți o fereastră de command prompt și executați `ipconfig/all`. Utilizați informațiile afișate pe ecran pentru a completa următorul tabel.

Setare	Valoare
DHCP e activate	
Adresa IP	
Masca de subrețea	
Servere DNS	
Timp obținere adresă	
Timp expirare adresă	

3.2.2 Sistemele Linux

1. Încărcați driverul pentru placa de rețea.
2. Editați fișierul `/etc/sysconfig/network-scripts/ifcfg-eth0` și faceți modificările necesare pentru utilizarea DHCP.

Exemplu de fișier `ifcfg-eth0`:

```
DEVICE=eth0
USERCTL = no
BOOTPROTO=DHCP
ONBOOT=yes
IPADDR= addrIP
NETMASK=address
NETWORK=address
BROADCAST=address
```

3. Editați/creați fișierul `/etc/sysconfig/network` pentru utilizarea DHCP.

```
NETWORKING = yes
HOSTNAME = hostname
FORWARD_IPV4 = yes
GATEWAYDEV = gwdev
GATEWAY = gwIP
```

4. Executați comanda `/etc/rc.d/init.d/network restart`.
5. Verificați adresa IP și parametri primiți de la serverul DHCP executând comanda `/sbin/ifconfig`.

3.3 Obținerea unei adrese IP utilizând APIPA.

Nu începeți execuția pasului 3.3 până când instructorul nu va anunța oprirea serverului DHCP.

1. În command prompt executați `ipconfig /release` apoi apăsați tasta enter.
2. În command prompt executați `ipconfig /renew` apoi apăsați tasta enter.
3. Ce mesaj este afișat și ce indică acest mesaj? Vă așteptați să aveți asignată o adresă IP?

4. În command prompt executați `ipconfig /all` și apoi apăsați tasta enter. Utilizați informațiile afișate pe ecran pentru a completa următorul tabel.

Setare	Valoare
DHCP activate	
Autoconfigurare activată	
Adresa IP	
Masca de subrețea	
Servere DNS	

3.4 Configurarea TCP/IP pentru asignarea manuală a unei adrese IP

1. În Local Area Connection alegeți Properties.
2. Verificați dacă Internet Protocol (TCP/IP) este bifată ca opțiune și apoi alegeți Properties.
3. În noua fereastră deschisă alegeți opțiunea Use the following IP address și în IP address completați cu informațiile furnizate de instructor.
4. Verificați ca opțiunea Use the following DNS server address să fie bifată și în Preferred DNS Server completați cu informațiile furnizate de instructor.
5. Apăsați butonul OK și închideți fereastra TCP/IP Properties. Apăsați butonul OK pentru a închide fereastra Local Area Connection Properties.
6. În command prompt executați `ipconfig/all`. Utilizați informațiile afișate pe ecran pentru a completa următorul tabel.

Setare	Valoare
DHCP activate	
Adresa IP	
Masca de subrețea	
Servere DNS	

3.5. Testarea comenzilor arp, nslookup și dhclient

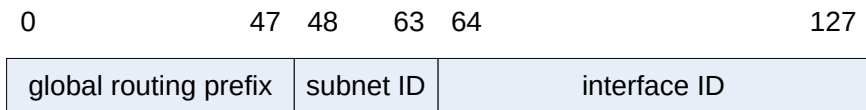
- 3.5.1. Folosind comanda arp vizualizați perechile de adrese MAC IP memorate.
- 3.5.2. Folosind comanda arp introduceți static o pereche de adrese MAC IP.
- 3.5.3. Folosind comanda nslookup setați un server DNS pentru interogare și vizualizați adresele IP corespunzătoare catorva site-uri.
- 3.5.4. Testați comanda dhclient (Linux)

IP versiunea 6

1. Consideratii teoretice

Reprezentarea adreselor IPv6 este realizata in hexazecimal utilizandu-se grupuri (campuri) de 16 biti separate prin “:”. Urmatoarele reguli permit scurtarea reprezentarii. Cei mai semnificativi digiti avand valoarea 0 dintr-un camp sunt optionali. Campuri successive de zerouri pot fi reprezentate ca si “::” aceasta prescurtare putand fi folosita o singura data intr-o adresa. O adresa nespecificata este scrisa ca si “::”.

Adresele unicast globale sunt formate din global routing prefix (48 biti), subnet ID (16 biti) si interface ID (64 biti).



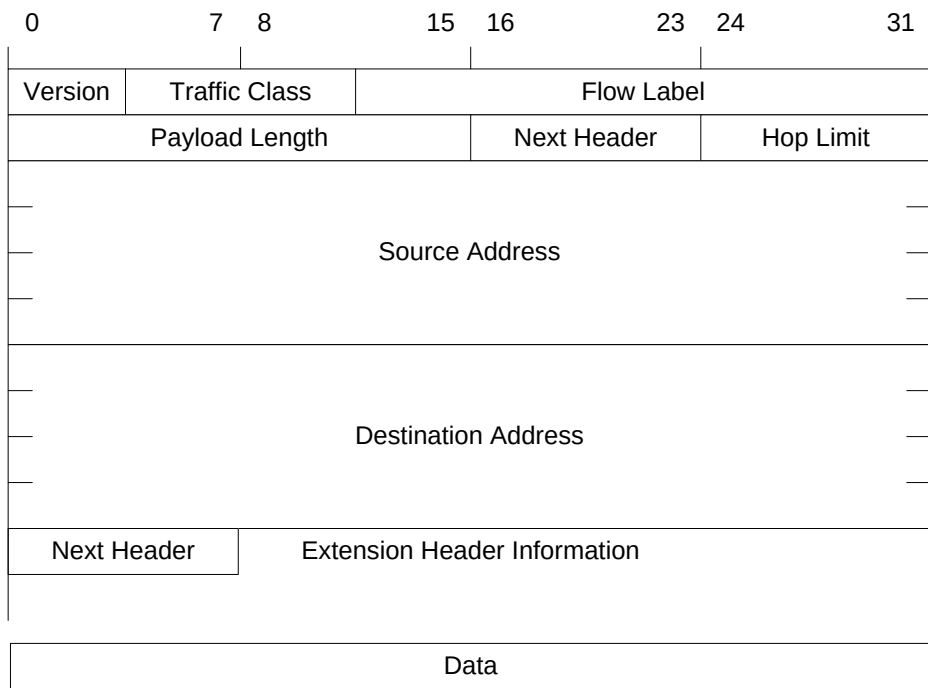
Formatul adresei IPv6 unicast globale

Adresele private sunt locale unui link sau site si nu sunt rutate in exterior. Valoarea primului octet este "FE" si urmatorul digit hexazecimal este in gama "8"- "F". Aceste adrese sunt impartite in doua tipuri: site-local si link-local. Adresele site-local au ca si scop un intreg site sau organizatie. Valoarea celui de-al treilea digit hexazecimal are este in gama "C"- "F". Adresele link-local au ca si scop o legatura fizica. Valoarea celui de-al treilea digit hexazecimal are este in gama "8"- "B".

Adresa loopback este “::1”. Adresa nespecificata, “::”, este folosita in campul sursa al unei datagrame transmisa de catre un dispozitiv care incearca sa isi configureze adresa IP.

Headerul IPv6

Headerul de baza al pachetului IPv6 are 8 campuri cu o lungime totala de 40 octeti.



Asignarea adreselor IPv6

Adresele IP pot fi asignate static sau dinamic. Asignarea statica poate fi realizata folosind un interface ID manual sau un interface ID EUI-64. Asignarea dinamica se poate face prin autoconfigurare stateless sau prin DHCP pentru IPv6.

Asignarea statica folosind un interface ID manual presupune atribuirea manuala atat a prefixului (network) cat si a interface ID-ului (host). Configurarea unei adrese IPv6 pentru interfetele echipmentelor Cisco se face cu comanda:

```
Router(config-if)#ipv6 address ipv6-address/prefix-length
```

Exemplu de configurare:

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface fastEthernet 0/0
Router(config-if)#ipv6 address 2001:C30:20:2:209:7CFF:FE4D:1501/64
Router(config-if)#no shutdown
```

Asignarea statica folosind un interface ID EUI-64 presupune atribuirea manuala a prefixului (network) si derivarea interface ID-ului (host) din adresa MAC a dispozitivului, cunoscuta ca si EUI-64 interface ID. Configurarea unei adrese IPv6 pentru interfetele echipmentelor Cisco se face cu comanda:

```
Router(config-if)#ipv6 address ipv6-prefix/prefix-length eui-64
```

Exemplu de configurare:

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface fastEthernet 0/0
Router(config-if)#ipv6 address 2001:C30:20:2::/64 eui-64
Router(config-if)#no shutdown
```

Comanda `ipv6 address` poate configura o adresa IPv6 globala. Adresa link-local este configurata automat cand o adresa este asignata interfeței.

Asignarea dinamica prin autoconfigurare stateless configureaza automat adresa IPv6 folosind informatiile din mesajele router advertisement (RA). Pentru asignarea dinamica prin autoconfigurare stateless lungimea prefixului transmis din mesajele RA trebuie sa fie de 64 biti. Extensia default router preference (DRP) furnizeaza o metrica preferentiala (high, medium, low) pentru ruterele default (gateway). DRP-ul rutelui default este semnalizat in mesajele RA. Mesajele router advertisement sunt transmise de la fiecare interfata a rutelilor, periodic si ca si raspuns la mesajele router solicitation. Mesajele router solicitation sunt transmise de catre hosturi la pornirea sistemului. Configurarea extensiei DRP pentru interfetele echipmentelor Cisco se face cu comanda:

```
Router(config-if)#ipv6 nd router-preference {high | medium | low}
```

Exemplu de configurare:

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface fastEthernet 0/0
Router(config-if)#ipv6 nd router-preference high
```

Asignarea dinamica prin DHCP pentru IPv6 presupune transmiterea parametrilor de configurare nodurilor IPv6 de la serverele DHCP prin protocolul DHCPv6 (stateful). Configurarea DHCPv6 Configuration Pool pentru echipamentele Cisco se face cu comenzile:

```
Router(config)#ipv6 dhcp pool poolname
Router(config-dhcp)#domain-name domain
Router(config-dhcp)#dns-server ipv6-address
Router(config-dhcp)#prefix-delegation ipv6-prefix/prefix-length client-duid [iaid iaidd]
[lifetime]
Router(config-dhcp)#prefix-delegation pool poolname [lifetime valid-lifetime preferred-
lifetime]
Router(config-if)#ipv6 dhcp server poolname [rapid-commit] [preference value] [allow-
hint]
```

Vizualizarea DHCPv6 unique identifier (DUID) se face cu comanda:

```
Router#show ipv6 dhcp
```

Exemplu de configurare:

```
Router>enable
Router#show ipv6 dhcp
This device's DHCPv6 unique identifier (DUID): 0003000100097C4D1501
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)# ipv6 dhcp pool rc
Router(config-dhcp)# domain-name rc.ro
Router(config-dhcp)# dns-server 2001:C30:20:2::1:1:1:1
Router(config-dhcp)#prefix-delegation 2001:C30:20:2::/64 0003000100097C4D1501
Router(config-dhcp)#exit
Router(config)#interface fastEthernet 0/0
Router(config-if)# ipv6 dhcp server rc
```

Vizualizarea adreselor ipv6 se face cu comanda:

```
Router#show ipv6 interface [brief] [type number] [prefix]
```

Strategii de tranzitie IPv6

Cele mai comune tehnici de tranzitie de la IPv4 la IPv6 sunt dual stacking si tunneling.

In dual stacking echipamentele sunt configurate pentru a suporta ambele protocoale, IPv6 fiind protocolul preferat.

Cele mai utilizate tehnici de tunneling sunt: manual IPv6-over-IPv4 tunneling si dynamic 6to4 tunneling. In cazul manual IPv6-over-IPv4 tunneling pachetele IPv6 sunt incapsulate in

protocolul IPv4. Dynamic 6to4 tunneling stabileste automatic conexiunea retelelor IPv6 prin retele IPv4.

Rutarea cu IPv6

IPv6 foloseste rutarea bazata pe potrivirea celui mai lung prefix (longest prefix match routing). Protocoalele de rutare pentru IPv6 sunt versiuni modificate ale celor mai uzuale protocoale de rutare.

Activarea IPv6 pe un ruter Cisco se face in doi pasi. Primul pas presupune activarea inaintarii traficului IPv6 iar al doilea pas este configurarea fiecarei interfete care necesita IPv6. Activarea inaintarii traficului IPv6 intre interfete se face cu comanda:

```
Router(config)#ipv6 unicast-routing
```

Exemplu de configurare:

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ipv6 unicast-routing
```

Configurarea unei rute statice pentru echipmentele Cisco se face cu comanda:

```
Router(config)#ipv6 route ipv6-prefix/prefix-length {ipv6-address | interface-type interface-number [ipv6-address]} [administrative-distance] [administrative-multicast-distance | unicast | multicast] [tag tag]
```

Exemplu de configurare:

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)# ipv6 route 2001:A30:10:2::/64 2001:C30:20:1:290:2BFF:FE71:6702
```

Vizualizarea tabelii de rutare pentru echipmentele Cisco se face cu comanda:

```
Router#show ipv6 route [ipv6-address | ipv6-prefix/prefix-length | protocol | interface-type interface-number]
```

Vizualizarea si setarea informatiilor

Pentru sistemul de operare Windows vizualizarea si setarea informatiilor se face cu ajutorul comenzilor ipconfig, route, netstat, netsh. Pentru sistemul de operare Linux vizualizarea si setarea informatiilor se face folosind comenzile ifconfig, route, netstat.

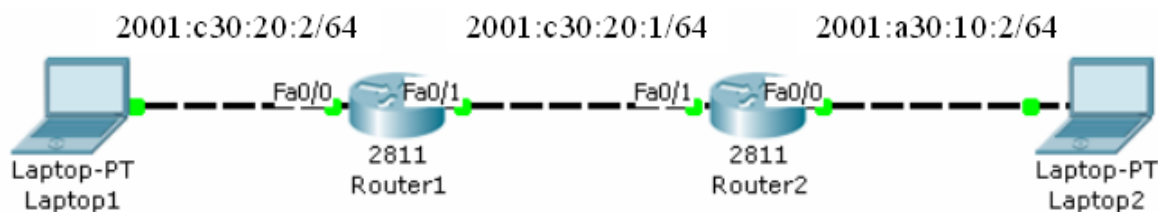
Interogarea DNS pentru sistemele de operare Windows si Linux se face cu ajutorul comenzii nslookup.

Testarea conectivitatii

Pentru sistemul de operare Linux si echipamentele Cisco testarea conectivitatii se face cu ajutorul comenzilor ping si traceroute. Pentru sistemul de operare Windows testarea se face folosind comenzile ping si tracert.

2. Desfasurarea lucrarii

1. Se vor testa pe statiile din laborator comenzile pentru vizualizarea si setarea informatiilor, interogarea DNS si testarea conectivitatii.
2. Folosindu-se simulatorul Packet Tracer se va configura retea de mai jos.



- a. Adresele interfetelor hosturilor se vor asigna static
- b. Adresele interfetelor ruterele se vor asigna static folosind interface ID EUI-64.
- c. Pentru rutere se vor configura rute default.
- d. Se vor vizualiza informatiile referitoare la adresele IPv6, tabelele de rutare IPv6 si se va testa conectivitatea.
- e. Pentru ruterele 1 si 2 se vor sterge rutele default si se vor configura rute pentru retelele 2001:a30:10:2/64 si respectiv 2001:c30:20:2/64.
- f. Se vor vizualiza informatiile referitoare la adresele IPv6, tabelele de rutare IPv6 si se va testa conectivitatea.

LUCRAREA NR. 8

REȚELE LAYER 2 ȘI PROTOCOLUL SPANNING-TREE

1. Scopul lucrării

Obiectivul acestei lucrări este descrierea switch-urilor și a bridge-urilor, descrierea configurării switch-urilor și cunoașterea protocolului Spanning-Tree.

2. Considerații teoretice

2.1 Switch-uri și bridge-uri

Switch-urile și bridge-urile sunt echipamente de nivel 2 folosite pentru creșterea lățimii de bandă disponibile și reducerea congestiei rețelei. Switch-urile și bridge-urile realizează două operații de bază: comutarea cadrelor de date și menținerea operațiilor de comutare. Switch-urile și bridge-urile segmentează rețeaua locală de calculatoare creînd multiple domenii de coliziune de dimensiune mai mici. Fiecare port crează un segment care este un domeniu de coliziune deoarece switch-ul sau bridge-ul învață adresele MAC a dispozitivelor conectate la fiecare port, introduce această informație într-o tabelă de switching sau bridging și înaintează sau blochează traficul pe baza acestei tabele. Segmentarea permite reducerea semnificativă a congestiei rețelei în fiecare segment. Dispozitivele dintr-un segment împart toată lățimea de bandă disponibilă. Dacă switch-ul sau bridge-ul nu știe unde să transmită cadrul, îl transmite broadcast la toate porturile. Când este returnat un răspuns, switch-ul sau bridge-ul înregistrează noua adresă în tabela de switching sau bridging. Un alt avantaj al conexiunii comutate este că permite Ethernet full-duplex care permite transmiterea unui pachet și recepția unui alt pachet în același timp. Dezavantajul echipamentelor de nivel 2 este că înaintează cadrele broadcast spre toate dispozitivele conectate din rețea astfel încât toate hosturile conectate la switch sau bridge sunt încă parte din același domeniu de broadcast.

Comutarea este clasificată ca și simetrică sau asimetrică. Comutarea simetrică furnizează conexiuni comutate între porturi cu aceeași lățime de bandă. Comutarea asimetrică furnizează conexiuni comutate între porturi cu lățime de bandă diferită. Comutarea asimetrică permite dedicarea unei lățimi de bandă mai mari pentru portul switch-ului conectat la server pentru prevenirea congestiei.

Modurile de comutare sunt clasificate ca și store-and-forward sau cut-through, fiecare mod reprezentând un compromis între latență și detectarea erorilor. În modul de comutare store-and-forward întregul cadru este recepționat înainte ca orice înaintare să aibă loc. Acest mod de comutare mărește latența transmisiei și permite creșterea detectării erorilor. În modul de comutare cut-through cadrul este înaintat prin switch înainte ca întregul cadru să fie recepționat. Trebuie citită cel puțin adresa destinație a cadrului înainte de a se putea transmite cadrul. Acest mod de comutare micșorează latența transmisiei și micșorează detectarea erorilor. Modul de comutare cut-through are două forme: fast-forward și fragment-free. Comutarea fast-forward înaintează pachetul după citirea adresei destinație. Acest mod de comutare are cel mai coborât nivel de latență și detectare a erorilor. Comutarea fragment-free înaintează pachetul după citirea primilor 64 octeți ai cadrului. Deoarece fragmentele rezultate în urma coliziunilor sunt mai mici de 64 octeți, modul de comutare fragment-free elimină

acest tip de eroare care reprezintă deasemenea majoritatea erorilor pachetelor. Acest mod de comutare are un nivel mai ridicat de latență și detectare a erorilor decât modul fast-forward.

2.2 Protocolul Spanning-Tree

Topologiile de rețea redundante măresc fiabilitatea rețelei prin introducerea legăturilor redundante. Aceste conexiuni introduce bucle fizice în rețea. Deoarece nivelul 2 nu are un mecanism de eliminare a cadrelor pierdute, cadrele se pot circula la nesfârșit într-o topologie cu bucle de nivel 2 determinând apariția a două tipuri de probleme: furtuna broadcast și instabilitatea tabelului de switching sau bridging. Furtuna broadcast este creată de cadrele broadcast inundate la nesfârșit spre toate porturile switch-urilor sau bridge-urilor, risipind lățimea de bandă sau făcând inutilizabilă rețeaua. Instabilitatea tabelului de switching sau bridging apare atunci când copii multiple ale unui cadru sosesc la porturi diferite ale unui switch sau bridge cauzând instabilitatea intrărilor MAC în tabela de switching sau bridging. Protocolul Spanning-Tree IEEE 802.1d folosește algoritmul spanning-tree pentru a crea o topologie logică bazată pe calea cea mai scurtă, fără bucle, într-o topologie cu bucle de nivel 2. Protocolul Rapid Spanning-Tree IEEE 802.1w folosește algoritmul rapid spanning-tree pentru a realiza aceeași funcție ca și algoritmul spanning-tree cu un timp de convergență mai mic.

Protocolul Spanning-Tree folosește mesaje de nivel 2 multicast Bridge Protocol Data Unit (BPDU) care sunt trimise de către dispozitivele de rețea la fiecare 2 secunde implicit. Structura acestor mesaje este prezentată în figura 9.1.

Root BID	Root Path Cost	Sender BID	Port ID
----------	----------------	------------	---------

Figura 9.1 Structura mesajului BPDU

BID este un camp de 8 octeți. Cei mai semnificativi 2 octeți reprezintă prioritatea bridge-ului sau switch-ului care are valoarea implicită de 32768 și cei mai puțin semnificativi 6 octeți reprezintă adresa MAC a bridge-ului sau switch-ului. Structura câmpului BID este prezentată în figura 9.2.

7	6	5	4	3	2	1	0
Bridge Priority		MAC address					

Figura 9.2 Structura câmpului BID

Protocolul Spanning-Tree calculează rețeaua cu calea cea mai scurtă pe baza costurilor legăturilor cumulate. Costurile legăturilor sunt bazate pe viteza legăturii. Câteva costuri ale legăturii pentru legături depășind 1 Gbps date în specificațiile IEEE 802.1d sunt prezentate mai jos.

Tabelul 9.1

Viteza legăturii	Cost
4Mbps	250
10Mbps	100
16Mbps	62
100Mbps	19
1Gbps	4
10Gbps	2

Câteva costuri ale legăturii pentru legături de 10 Gbps sau mai mult date în specificațiile IEEE 802.1d sunt prezentate în tabelul următor.

Tabelul 9.2

Viteza legăturii	Cost
10Mbps	2000000
100Mbps	200000
1Gbps	20000
10Gbps	2000
1Tbps	20
10Tbps	2

Port ID este un câmp de 2 octeți. Cel mai semnificativ octet este prioritatea portului care are valoarea implicită de 128 și cel mai puțin semnificativ octet este numărul portului. Structura Port ID este prezentată în figura 9.3.

1	0
Prioritatea portului	Numărul portului

Figura 9.3 Structura Port ID

Protocolul Spanning-Tree stabilește un singur nod rădăcină, numit bridge rădăcină și construiește o topologie care are o cale pentru atingerea fiecărui nod de rețea. Arborele rezultat își are originea în bridge-ul rădăcină. Bridge-urile și switch-urile calculează cea mai scurtă cale de la ele spre bridge-ul rădăcină. Prima decizie pe care o iau toate bridge-urile sau switch-urile din rețea este identificarea bridge-ului rădăcină, care se realizează cu ajutorul mesajelor BPDU recepționate de către toate bridge-urile și switch-urile. Toate celelalte decizii din rețea sunt luate în funcție de acest bridge rădăcină. Când este pornit un bridge sau switch, acesta presupune că este bridge-ul rădăcină și transmite mesaje BPDU conținând adresa MAC a bridge-ului sau switch-ului atât în câmpul root BID cât și în câmpul sender BID. Dacă un bridge sau switch recepționează un BPDU cu o valoare root BID mai mică setează acest root BID în mesajele BPDU care sunt transmise. Bridge-ul sau switch-ul cu cea mai mică valoare BID va fi bridge-ul rădăcină. Setarea priorității bridge-ului sau switch-ului la o valoare mai mică decât cea implicită va face valoarea BID mai mică și va influența identificarea bridge-ului rădăcină. Pentru fiecare segment LAN, protocolul Spanning-Tree stabilește un switch desemnat ca și cel mai apropiat de bridge-ul rădăcină care se ocupă de toată comunicația din acel LAN spre bridge-ul rădăcină. Pentru fiecare bridge diferit de bridge-ul rădăcină este ales un port rădăcină, care oferă cea mai bună cale spre bridge-ul rădăcină. Astfel, portul cu cel mai mic cost al căii spre bridge-ul rădăcină este ales ca și port

rădăcină. Dacă mai multe porturi au același cost al căii spre bridge-ul rădăcină, portul cu cel mai mic Port ID este selectat ca și root port. Protocolul Spanning-Tree selectează de asemenea porturile desemnate care sunt parte din arborele cu cea mai scurtă cale. Astfel, portul cu cel mai mic cost al căii spre bridge-ul rădăcină este selectat ca și port desemnat. Dacă mai mult de un port în segment are același cost al căii, portul pe care bridge-ul sau switch-ul are cel mai mic bridge sau switch ID este selectat ca și port desemnat. Pe bridge-ul rădăcină, toate porturile acestuia sunt porturi desemnate. Căile redundante care nu sunt parte a arborelui cu cea mai scurtă cale sunt blocate și cadrele de date recepționate pe legăturile blocate sunt eliminate.

Fiecare port pe un bridge sau switch care folosește protocolul Spanning-Tree are una din următoarele cinci stări: blocare, ascultare, învățare, înaintare și dezactivat. În starea de blocare porturile pot doar primi mesaje BPDU, cadrele de date sunt eliminate și adresele nu pot fi învățate. Schimbarea din această stare poate dura până la 20 secunde. Porturile trec din starea de blocare în starea de ascultare. În această stare, switch-urile sau bridge-urile determină dacă sunt alte căi spre bridge-ul rădăcină. Calea care nu are cel mai mic cost al căii spre bridge-ul rădăcină se întoarce în starea de blocare. În starea de ascultare mesajele BPDU sunt procesate, datele utilizatorului nu sunt înaintate și adresele MAC nu sunt învățate. Perioada de ascultare este numită întârzierea înaintării și durează 15 secunde. Porturile trec din starea de ascultare în starea de învățare. În această stare mesajele BPDU sunt procesate, datele utilizatorului nu sunt înaintate, dar adresele MAC sunt învățate de la orice trafic care este văzut. Starea de învățare este de asemenea numită întârzierea înaintării și durează 15 secunde. Un port trece din starea de învățare în starea de înaintare. În această stare mesajele BPDU sunt procesate, datele utilizatorului sunt înaintate și adresele MAC continuă să fie învățate. Portul poate fi în starea dezactivat când este blocat administrativ sau există o defecțiune. Valorile de timp date pentru fiecare stare sunt valorile implicite. Aceste valori au fost calculate presupunând că vor fi maximum șapte switch-uri în fiecare ramură a arborelui de la bridge-ul rădăcină. Când topologia rețelei se schimbă, switch-urile și bridge-urile recalculază arborele. Convergența pe o nouă topologie spanning-tree folosind standardul IEEE 802.1d poate dura până la 50 secunde.

3. Desfășurarea lucrării

3.1 Configurarea switch-urilor

Conectați un port COM al calculatorului la portul de Consolă al Switch-ului folosind un cablu rollover RJ-45-la-RJ-45 și un adaptor RJ-45-la-DB9 sau RJ-45-la-DB25 funcție de tipul portului COM.

Intrați în modul User EXEC cu programul HyperTerminal folosind un nume de conexiune, portul COM corespunzător și următoarele setări ale portului COM: 9600 biți pe secundă, 8 biți de date, fără paritate, 1 bit de stop și controlul fuxului hardware.

Switch-urile și router-ele Cisco folosesc o interfață de tip linie de comandă (command-line interface - CLI) foarte asemănătoare care este folosită pentru configurare și verificare.

Comanda `help` este semnul întrebării (?) care afișează lista comenzilor disponibile pentru modul de comandă curent, lista comenzilor care încep cu o anumită secvență de caractere sau lista cuvintelor cheie sau argumentelor care sunt asociate cu o anumită comandă.

Switch-urile prezintă mai multe moduri de comandă. Modul User EXEC are un set de comenzi limitat care schimbă setările terminalului, execută teste de bază sau afișează informațiile legate de sistem. Comanda `enable` este folosită pentru a schimba din modul User EXEC în modul Privileged EXEC. Modul Privileged EXEC prezintă un set de comenzi mai mare care include setul de comenzi al modului User EXEC și comanda `configure` folosită pentru a schimba din modul Privileged EXEC în modul global configuration. Modul global configuration permite accesarea altor moduri de comandă, care sunt folosite pentru configurarea switch-ului. Comanda `exit` este folosită pentru a ieși dintr-un mod de comandă.

Executați comanda `help` pentru a vizualiza lista comenzilor disponibile pentru modul User EXEC.

Executați comanda `show version` pentru a vizualiza informațiile legate de versiune pentru software și hardware.

Executați comanda `show running-config` pentru a vizualiza fișierul de configurare curent al switch-ului.

Intrați în modul Privileged EXEC cu ajutorul comenzii `enable`.

Executați comanda `copy running-config startup-config` pentru a copia fișierul de configurare curent în fișierul de configurare back up.

Pentru a șterge complet configurația switch-ului trebuie să parcurși următorii pași:

- ștergeți fișierul conținând baza de date VLAN `vlan.dat` din directorul flash cu comanda `delete flash:vlan.dat`;
- ștergeți fișierul de configurare back up `startup-config` cu comanda `erase startup-config`
- restartați switch-ul cu comanda `reload`.

Schimbați din modul Privileged EXEC în modul global configuration cu comanda `configure terminal`.

Setați numele switch-ului cu comanda `hostname numele_hostului`.

Configurați linia terminal primară cu următoarele comenzi:

- `line console 0`
- `password parola`
- `login`
- `exit`

Configurați terminalul virtual cu următoarele comenzi:

- `line vty 0 4`
- `password parola`
- `login`
- `exit`

Pentru a permite accesul la switch prin aplicații TCP/IP, trebuie setată o adresă IP și un default gateway. Aceasta permite configurarea switch-ului folosind o conexiune telnet. Configurați o adresă IP și un default gateway cu următoarele comenzi:

- `interface VLAN1`
- `ip address adresa_ip netmask`
- `no shutdown`
- `exit`
- `ip default-gateway adresa_default_gateway`

3.2 Protocolul Spanning-Tree

Cablați rețeaua prezentată în the figura 9.4.

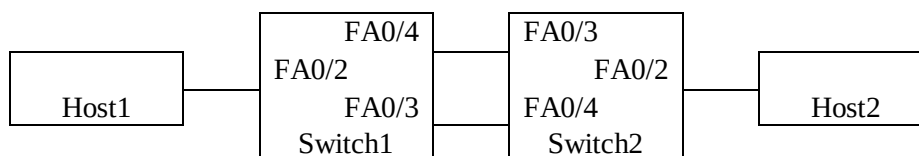


Figura 9.4 *Rețea test Spanning-Tree*

1. Verificați conectivitatea dintre Host1 și Host2 cu comanda `ping`.
2. Conectați-vă la Switch1 și intrați în modul Privileged EXEC.
3. Vizualizați informațiile legate de Spanning-Tree cu comanda `show spanning-tree`. Examinați și explicați ieșirea acestei comenzi.
4. Conectați-vă la Switch1 și intrați în modul Privileged EXEC.
5. Executați comanda `show spanning-tree`.
6. Conectați-vă la switch-ul care nu este rădăcină și intrați în modul global configuration.
7. Schimbați switch-ul rădăcină prin schimbarea priorității la o valoare mai mică decât valoarea implicită cu comanda `spanning-tree vlan număr_vlan priority număr_prioritate`. Număr_vlan este 1.
8. Ieșiți în modul Privileged EXEC.
9. Executați comanda `show spanning-tree`.
10. Conectați-vă la celălalt switch și intrați în modul Privileged EXEC.
11. Executați comanda `show spanning-tree`.
12. Conectați-vă la switch-ul rădăcină și intrați în modul global configuration.
13. Intrați în modul interface cu comanda `interface fastethernet număr_interfață`. Număr_interfață este 0/4.
14. Schimbați portul rădăcină prin schimbarea priorității la o valoare mai mică decât valoarea implicită cu comanda `spanning-tree vlan număr_vlan port-priority număr_port-priority`.
15. Ieșiți în modul Privileged EXEC.
16. Executați comanda `show spanning-tree`.
17. Conectați-vă la switch-ul care nu este rădăcină și intrați în modul Privileged EXEC.
18. Executați comanda `show spanning-tree`.
19. Deconectați legătura dintre Switch1 și Switch2 care este conectată la portul rădăcină al switch-ului care nu este rădăcină.

20. Executați comanda `show spanning-tree` de mai multe ori până când portul 0/3 trece în starea de înaintare.
21. Conectați-vă la switch-ul rădăcină și intrați în modul Privileged EXEC.
22. Executați comanda `show spanning-tree`.

Notițe

LUCRAREA NR. 9

REȚELE LOCALE VIRTUALE ȘI TRUNKING

1. Obiective

Obiectivul acestei lucrări de laborator este definirea, clasificarea și identificarea VLAN-urilor precum și avantajele oferite de utilizarea acestora.

2. Considerații teoretice

2.1 Definiția și avantajele unui VLAN

O facilitate importantă oferită de comutarea Ethernet și nu numai este cea de VLAN. Un VLAN este o partiție a mulțimii de dispozitive conectate în rețeaua locală. Partițiile sunt mutual disjuncte. Gruparea în VLAN-uri se poate face după diferite criterii cum ar fi: funcții, locație, aplicație, etc. Această grupare se poate face indiferent de locația fizică a dispozitivelor. Dispozitivele componente a unui VLAN sunt restricționate la a comunica doar cu dispozitive din același VLAN. Conectivitatea între VLAN-uri este facilitată de rutere. VLAN-urile funcționează prin segmentarea logică a rețelei în domenii de broadcast, fiecare VLAN reprezentând un domeniu de broadcast diferit. Comutatorul, switch-ul, menține câte o tabelă de bridging diferită pentru fiecare VLAN în parte. Beneficiile VLAN-urilor sunt creșterea performanței rețelei, scalabilitate mărită, creșterea gradului de securitate și un management mai performant. Un ruter într-o rețea locală cu facilități de VLAN permite filtrarea pachetelor de broadcast, liste de acces și managementul traficului.

2.2 Clasificare și identificare VLAN

Există două tipuri de apartenență la VLAN: static și dinamic. Apartenența statică se mai numește și bazată pe port sau portcentrică. În acest tip apartenență unui port la un VLAN se configurează de către administrator și toți utilizatorii acestui port sunt asigurați VLAN-ului configurat. Acest tip de apartenență este simplu și ușor de configurat și este utilizat în rețele în care mișcarea dispozitivelor este rară și controlată. VLAN-ul implicit este VLAN-ul 1. Acesta nu poate fi șters și la inițializare toate porturile aparțin acestui VLAN. VLAN-urile dinamice sunt create prin soft de management. În acest tip de VLAN asignarea unui port la un anumit VLAN se face pe baza adreselor MAC a dispozitivului conectat la switch. La conectare comutatorul interoghează o bază de date care conține asignarea la VLAN pentru adresa MAC dată. Pot fi conectate mai multe dispozitive la un port dinamic dacă toate aparțin aceluiași VLAN. VLAN-uri dinamice sunt utilizate în medii cu relocări frecvente și necontrolate de dispozitive.

La utilizarea legăturilor între comutatoare cu VLAN-uri multiple antetul cadrului este modificat pentru a reflecta apartenența la un anumit VLAN. Înaintea trimiterii cadrului la dispozitivul final antetul cadrului este refăcut la forma inițială. Protocolul IEEE 802.1Q este un standard IEEE pentru identificarea VLAN-urilor prin înserarea unui identificator de VLAN în antetul cadrului și recalcularea CRC. Acest procedeu se numește marcare cadru sau

marcare internă și este cel mai răspândit mod de marcare. Structura cadrului este prezentată în figura 8.1.

Adresă MAC originală	2-byte TPID	2-byte TCI	Type/Data inițiale	CRC nou
-------------------------	----------------	---------------	-----------------------	------------

Figura 8.1 Cadru 802.1Q

TPID (*Tag Protocol Identifier*) indică un cadru de tip 802.1Q/802.1p și are valoarea fixă de 0x8100.

TCI (*Tag Control Information*) conține următoarele câmpuri: 3-biți prioritate utilizator, 1-bit indicator format canonic (CFI) și 12-biți identificator VLAN (VLANID) - identifică în mod unic VLAN-ul de care aparține cadrul.

Pentru a facilita comunicarea între VLAN-uri cu ajutorul unui ruter fiecărui VLAN i se va asigura o adresă de rețea distinctă.

2.3 Trunking

Comunicația între switchuri cu mai multe VLAN-uri poate fi realizată în două moduri. Prima metodă necesită câte o legătură pentru fiecare VLAN. Această metodă nu este scalabilă deoarece necesită prea multe porturi pentru interconectare. A doua metodă necesită o legătură de tip trunk între comutatoare. O legătură de tip TRUNK este o legătură punct-la-punct care suportă mai multe VLAN-uri. Un port care formează o legătură de tip TRUNK nu aparține nici unui VLAN anume. Pentru a identifica cadrele peste legăturile de tip TRUNK se utilizează protocolul IEEE 802.1Q.

3. Desfășurarea lucrării

3.1 Crearea și asignarea de VLAN-uri

Cablați rețeaua din figura 8.2.

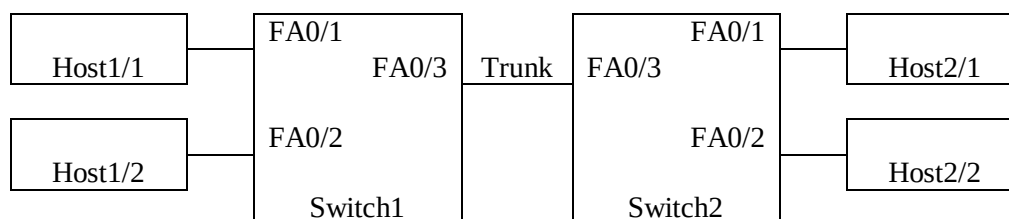


Figura 8.2 Rețea test VLAN

Pentru VLAN1 și VLAN2 considerați 172.27.0.0/16 respectiv 172.28.0.0/16 ca adrese de rețea.

Asignați la fiecare host o adresă IP din VLAN1.

Conectați-vă la Switch1, intrați în modul Privileged EXEC și stergeți configurația existentă.

Conectați-vă la Switch1, intrați în modul Privileged EXEC și vizualizați informația despre VLAN cu comenzile `show vlan` sau `show vlan brief`.

Pentru a vizualiza informații legate de un singur VLAN utilizați comenzile: `show vlan id vlan_number` sau `show vlan name vlan_name`.

Verificați conectivitatea între hosturile conectate la Switch1 cu comanda `ping`.

Conectați-vă la Switch1 și intrați în modul de configurare globală.

Intrați în modul config-vlan cu comanda `vlan vlan_number`. Vlan_number este 2 și este VLAN-ul ce va fi creat.

Numiți VLAN-ul creat cu comanda `name vlan_name`.

Reveniți în modul Privileged EXEC cu comanda `end`.

Tastați comanda `show vlan`.

Verificați conectivitatea între hosturile conectate la Switch1.

Asignați la Host1/2 o adresă IP din VLAN2.

Intrați în modul de configurare globală.

Intrați în modul de configurare interfață. Selectați interfața Fa0/2.

Modificați portul din starea default în modul acces cu comanda `switchport mode access`.

Re-assignați portul din VLAN1 în VLAN2 cu comanda `switchport access vlan vlan_number`.

Reveniți în modul Privileged EXEC cu comanda `end`.

Tastați comanda `show vlan`.

Verificați conectivitatea între hosturile conectate la Switch1.

Urmați pașii similari pentru Switch2 și Host 2/2 pentru crearea VLAN2, re-assignarea portului 0/2 la acest VLAN și asignarea unei adrese IP din VLAN2 la Host2/2.

3.2 Implementarea trunking

Verificați conectivitatea între hosturi care aparțin aceluiași VLAN din switchuri diferite.

Conectați-vă la Switch2, intrați în modul Privileged EXEC și afișați informația despre trunking cu comanda `show interfaces trunk`.

Intrați în modul de configurare globală.

Intrați în modul de configurare interfață. Selectați interfața de tip trunk.

Setați portul din modul trunking în mod acces cu comanda `switchport mode access`.

Reveniți în modul Privileged EXEC și tastați comanda `show vlan`.

Tastați comanda `show interfaces trunk`.

Verificați conectivitatea între hosturi care aparțin aceluiași VLAN din switchuri diferite.

Intrați în modul de configurare globală.

Intrați în modul de configurare interfață. Selectați aceeași interfață.

Setați portul din modul trunking în mod acces cu comanda `switchport mode trunk`.

Reveniți în modul Privileged EXEC și tastați comanda `show vlan`.

Tastați comanda `show interfaces trunk`.

Verificați conectivitatea între hosturi care aparțin aceluiași VLAN din switchuri diferite.

3.3 Ștergerea VLAN-urilor

Ștergeți VLAN2 de pe o interfață cu forma `no` a comenzii de asociere.

Tastați comanda `show vlan`.

Verificați conectivitatea între hosturi.

Ștergeți VLAN2 din VLAN database cu forma `no` a comenzii de creare.

Tastați comanda `show vlan`.

Verificați conectivitatea între hosturi.

Ștergeți orice informație despre VLAN-urile create prin ștergerea VLAN database cu comanda `delete flash:vlan.dat`.

Tastați comanda `show vlan`.

Verificați conectivitatea între hosturi.

Notițe

LUCRAREA NR. 7

PROGRAMARE ÎN REȚEA FOLOSIND SOCKETURI

1. Scopul lucrării

Lucrarea își propune prezentarea noțiunilor care stau la baza mecanismelor transmisiei de date sub formă de mesaje între procese tip client - server bazată pe socketuri (Unix sau în Internet.) cât și prezentarea diverselor modalități de comunicare client – server, asimilarea funcțiilor principale de bibliotecă (socketurile Unix, sau Internet) utilizate în proiectarea aplicațiilor de tip client – server.

2. Considerații teoretice

2.1 Programarea cu socketuri. Concepte.

Există diverse tipuri de socketuri, diferențiate după topologia comunicației respectiv a tipurilor de adrese utilizate și anume adrese Internet (în mod corespunzător socketurile se numesc socketuri Internet), căi către noduri locale pentru comunicarea între procese situate pe aceeași mașină (socketuri Unix) sau adrese tip X.25 (socketuri X.25, mai puțin utilizate).

Cateva **concepte specifice** implementării mecanismelor de comunicație de date utilizând socketurile vor fi detaliate în cele ce urmează.

Socket - reprezintă un capăt de comunicație, definind o formă de comunicare între procese (Inter Process Communication) situate pe aceeași mașină sau pe mașini diferite conectate printr-o rețea. O pereche de socketuri conectate realizează o interfață de comunicație între două procese, interfață similară celei de tip pipe din Unix.

Asocierea- definește în mod unic conexiunea stabilită între două capete de comunicație, ca fiind un set de parametri de forma (*protocol_local*, *adresa _locală*, *port_local*, *adresa_remote*, *port_remote*). Noțiunile de *port_sursă*, respectiv *port_destinație* permit definirea unică a comunicației între perechi de procese și realizează identificarea unică a unui SAP (Service access point) de nivel transport.

Descriptor de socket - reprezintă un descriptor de fișier, este argumentul utilizat în funcțiile de bibliotecă.

Binding – operația prin care se asociază o adresă de socket unui socket, pentru ca acesta să poată fi accesat.

Portul – reprezintă un identificator folosit pentru a diferenția socketurile aflate la aceeași adresă. Pot fi folosite adrese cuprinse între 1024- 49151 (porturi înregistrate), respectiv adrese între 49152 – 65535, numite și porturi efemere sau dinamice, adresele între 1 și 1023 sunt rezervate de sistem(vezi fișierul etc/services din UNIX) .

Familie de adrese - reprezintă formatul de adrese folosit pentru a identifica adresele utilizate în socketuri(cele mai uzuale forme de adrese sunt adresele Internet și Unix).

Adresa de socket - este funcție de familia de adrese utilizate (pentru domeniul familiei de adrese Internet constă din adresa Internet și adresa portului utilizat în host.

Adresa Internet – reprezintă o adresă pe 4 octeți care identifică un nod (host sau ruter) într-o rețea.

2.2. Modelul client-server

Orice operație în rețea poate fi privită ca un proces client care comunică cu un proces server. Procesul server creează un socket, îi asociază o adresă și lansează un mecanism de ascultare a cererilor de conectare din partea clienților, procesul client crează la rândul său un socket și solicită conectarea la server. După acceptarea cererii de către server și stabilirea conexiunii, se poate stabili comunicația între socketuri.

Acest model clasic poate varia în funcție de natura aplicației proiectate, astfel el poate fi simetric sau asimetric, bazat pe comunicație simplex sau duplex. Bazat pe acest model client-server au fost proiectate o serie de aplicații uzuale și anume `telnet` (pentru conectarea la un host remote folosind portul 23) pentru care un program aflat în acel host, respectiv `telnetd`, va răspunde, `ftp/ftpd`, sau `bootp/bootpd`.

Lista funcțiilor uzuale din biblioteca de socketuri:

- `socket()` - creează un nou descriptor de socket;
- `bind()` – realizează legarea unei adrese și a portului corespunzător la socket;
- `connect()` - permite stabilirea unei conexiuni active cu un server remote
- `listen()` – ascultă cererile de conectare, este utilizată într-un *socket pasiv*;
- `accept()` – permite crearea unui nou socket corespunzător unei noi cereri de conectare;
- `send()`, `recv()`, `sendto()`, `recvfrom()` – transmite- recepționează *streamuri* sau *datagrame*;
- `close()`, `shutdown()`- închide o conexiune;
- `getpeername()` – determină numele hostului pereche din conexiune;
- `getsockname()` – determină numele socketului utilizat;
- `gethostbyname()`, `gethostbyaddr()`- deteremină numele sau adresa socketului din hostul specificat.

Comunicarea între procesele client, respectiv server se bazează pe apelul funcției `socket()`, apel care returnează un descriptor de socket. Acest descriptor este utilizat în apelurile diferitelor funcții specializate pentru transmisia de date (cum ar fi de exemplu `send()` și `recv()`). Detalii de sintaxa pentru aceste funcții se găsesc în anexa 1.

2.3. Socketuri de tip stream vs. socketuri datagramă

PROGRAMAREA ÎN REȚEA FOLOSIND SOCKETURI

Comunicația între programele client și server se desfășoară folosind protocoale de nivel *transport orientate conexiune (TCP)*, sau *protocoale neorientate conexiune (UDP)*.

Stream socketurile reprezintă fluxuri de comunicație sigure (fără erori), de tip full-duplex, bazat pe protocolul TCP ("Transmission Control Protocol") care asigură transmisia secvențială și fără erori a datelor.

Socketuri datagramă – sunt cunoscute și sub numele de socketuri fără conexiune, deoarece ele folosesc adresele IP pentru rutare, dar protocolul de nivel transport utilizat este UDP("User Datagram Protocol"). Aceste socketuri nu mențin o conexiune deschisă pe durata comunicației, ele realizând transferul pachet cu pachet (exemple de aplicații care utilizează socketurile datagramă sunt tftp (Trivial File Transfer Proticol), bootp ()).

2.4 Tipuri de date utilizate de interfața de socketuri

Structura următoare memorează adresa de socket pentru diferite tipuri de socketuri și este folosită în practica programării:

```
struct sockaddr
{
    unsigned short sa_family; /* familia de adrese, AF_XXX */
    char sa_data[14];        /* 14 bytes adresa de protocol */
};
```

Pentru a putea utiliza în programe structura struct sockaddr, a fost proiectată o structură paralelă: struct sockaddr_in definită în continuare astfel:

```
struct sockaddr_in
{
    short int  sin_family;      /*familia de adrese */
    unsigned short int  sin_port; /* număr port */
    struct in_addr sin_addr;    /* adresă internet */
    char sin_zero[8];          /* neutilizați */
};
```

iar

```
/* adresa Internet */
struct in_addr
{
    unsigned long s_addr;
};
```

Această structură permite o referire simplă a elementelor unei adrese de socket. Astfel sin_zero va fi setată la zero folosind funcțiile bzero() sau memset(). Pointerul la struct sockaddr_in mapează pointerul la structura struct sockaddr. Trebuie remarcat faptul că

`sin_family` corespunde la `sa_family` în structura `struct sockaddr` și va fi setată la `"AF_INET"` valoare utilizată pentru familia de protocoale TCP-IP.

2.5. Funcții de conversie

Deoarece în transmisia de date, pentru a proiecta aplicații portabile este necesară realizarea conversiei datelor transmise între hosturi care utilizează în mod diferit fie reprezentarea `little_endian`, fie cea `big_endian` și deoarece reprezentarea datelor la nivel de rețea este unică, (protocoalele din familia TCP-IP folosesc reprezentarea la nivel de rețea numită *Network Byte Order*) sunt necesare o serie de funcții care să realizeze aceste conversii.

Câteva din cele mai utilizate funcții de conversie în programarea cu socketuri sunt:

- `htons()` -- "Host to Network Short", conversie host la rețea short;
- `htonl()` -- "Host to Network Long", conversie host la rețea long;
- `ntohs()` -- "Network to Host Short", conversie rețea la host short;
- `ntohl()` -- "Network to Host Long", conversie rețea la host long.

Cunoscut fiind faptul că `sin_addr` și `sin_port` fiind încapsulate în pachetul IP, respectiv UDP-TCP, este necesar să fie în ordinea `Network_byte_order`, iar câmpul `sin_family` utilizat fiind de către kernelul sistemului de operare (pentru a determina ce tip de adresă conține structura), este în ordinea `Host Byte Order`, este necesară utilizarea funcțiilor de conversie.

Presupunem că analizăm structura `struct sockaddr_in ina`, și o adresă IP `"xxx.yyy.z.uu"` pe care dorim să o memorăm în structură. Funcția pe care o utilizăm este, `inet_addr()`, convertește o adresă IP din forma cunoscută (notația `numbers-and-dots`) într-un întreg fără semn astfel:

```
ina.sin_addr.s_addr = inet_addr("xxx.yyy.z.uu");
```

`inet_addr()` returnează adresa în `Network Byte Order`. Pentru tipărirea unei adrese în notația cunoscută cu punct, este necesară utilizarea funcției `inet_ntoa()` ("`ntoa`" reprezintă "network to ascii") așa cum se poate remarca în exemplul :

```
printf("%s",inet_ntoa(ina.sin_addr));
char *a1, *a2;
.
.
a1 = inet_ntoa(ina1.sin_addr);
a2 = inet_ntoa(ina2.sin_addr);
printf("address 1: %s\n",a1);
printf("address 2: %s\n",a2);
```

3. Desfășurarea lucrării

3.1 Rulați programul `listener` pe o mașină, apoi `talker` pe alta și urmăriți modul în care se desfășoară comunicația. Programul `talker` apelează funcția `connect()` și specifică adresa `listener`-ului pentru a trimite și recepționa. Codul sursă pentru programele de test se află la sfârșitul lucrării. Modificați-l corespunzător astfel încât să se poată desfășura comunicația în situația în care ambele programe rulează pe aceeași mașină fizică respectiv pe mașini diferite.

3.2. Proiectați trei programe de test `client1.c`, `client2.c` și `server.c` conectate într-un pipeline logic, astfel: programul `client1.c` transferă un mesaj de tip caracter procesului `server.c` care recepționează caracterul, realizează o prelucrare asupra lui și transmite procesului `client2.c` care tipărește rezultatul.

Modificați programele astfel încât valoarea transferată prin pipeline să fie consecutiv un întreg, respectiv o structură conținând valori de tip caracter și întreg.

3.3. Modificați programele proiectate anterior astfel încât să poată *funcționa și în domeniul Internet*. Dacă arhitecturile mașinilor din domeniu sunt diferite, deci structurile pentru mesaje sunt de dimensiuni diferite, proiectați un algoritm pentru a putea transmite structura.

3.4. Să se proiecteze un server simplu (de tip multiplexor), care să permită unui sistem de trei clienți conectați într-o topologie de tip stea să comunice între ei. Programul server acceptă conexiuni din partea oricărui client și va stabili conexiunile de tip socket cu fiecare dintre clienți. Serverul va tipări mesajul recepționat și apoi îl va transmite clientului destinație. Transmiterea mesajului (cu lungimea de 40 caractere) se va realiza la introducerea caracterului CR. Mesajul va conține identificatorii clienților sursă respectiv destinație (ID), iar clientul care recepționează mesajul îl va scrie în fișierul propriu de mesaje. Pentru fiecare client se va proiecta un meniu care să permită selecția uneia dintre opțiunile: alegere client cu care comunică, tipărire-ștergere mesaje curente aflate în fișierul de mesaje al clientului, ieșire cu ștergerea nodului-client din rețea.

3.5. Să se proiecteze un server de timp. Serverul trebuie să poată răspunde solicitărilor simultane a mai multor clienți, consultând o bază de date care conține data și ora exactă pentru zona solicitată.

3.6. Proiectați un sistem de tip client server care să permită transferul unui fișier de pe mașina server pe cea client.

Notițe

LUCRAREA NR. 8

PROGRAMARE ÎN REȚEA FOLOSIND SOCKETURI JAVA

1. Scopul lucrării

Lucrarea își propune studiul comunicației de tip client-server bazată pe mecanismele de tip socket, folosind facilitățile oferite de limbajul Java. Acest limbaj a fost ales datorită faptului că este un limbaj obiectual și oferă *mecanisme evaluate de comunicație în rețea, suport robust pentru concurență* și pentru tratarea erorilor. Se dorește familiarizarea cu mecanismele specifice și clasele (constructorii și metodele acestora) utilizate pentru comunicarea în rețea existente în limbajul Java (diverse categorii de socketuri, URL-uri) și proiectarea unor aplicații simple client – server.

2. Considerații teoretice

2.1 Introducere

Conceptele teoretice necesare programării în rețea folosind socketurile Unix se regăsesc și în cazul programării în rețea folosind limbajul Java. Pentru proiectarea aplicațiilor de rețea se folosește pachetul Java.net, care oferă suport independent de sistem pentru implementarea modelului de comunicație tip client - server. Acest pachet conține clase, interfețe și excepții ce implementează în formă obiectuală conceptele uzuale de comunicare.

Astfel, un client Java, pentru a obține un anumit serviciu de la un server remote, folosește adresa serverului pentru a solicita conexiunea, iar serverul ascultă rețeaua și așteaptă cereri din partea clienților. Deoarece un calculator posedă o singură conexiune fizică la rețea, conexiune utilizată de toate aplicațiile ce solicită lucrul în rețea, pentru identificarea aplicației căreia îi este destinat un pachet sunt folosite porturile. Stabilirea conexiunii presupune utilizarea ca și identificatori de hosturi comunicante a adresei IP și a adresei portului.

Pachetele *Java.net* și *Java.io* conțin clasele și metodele necesare programării în rețea. Clasele și metodele lor, necesare implementării programării cu socketuri în Java vor fi descrise în cele ce urmează. O listă deloc exhaustivă a acestora conține următoarele clase: DatagramPacket, DatagramSocket, MulticastSocket, InetAddress, ServerSocket, Socket, URL, URLConnection, URLEncoder, etc.

Instrumentele mediului Java utilizate în programarea și depanarea programelor sunt:

- java - interpretorul Java;
- javac - compilatorul Java;
- appletviewer - vizualizorul de appleturi Java;
- javadoc - generatorul de fișiere de documentație API;
- javap - dezasamblorul pentru fișiere class;
- jdb - debugger (Java debugger);
- javah - generatorul de fișiere C de tip header și de tip stub.

2.2 Elemente de bază în programarea cu socketuri. Clasa InetAddress, URL

Pentru a permite lucrul cu identificatori simbolici de tip nume dar și cu adresele IP ale hosturilor din rețea, în Java este definită clasa `InetAddress` ce furnizează abstracțiunile necesare accesării hosturilor.

Metodele statice ale clasei sunt:

- `Public static InetAddress getByName (String host) throws UnknownHostException` – returnează numele hostului local pentru care se cunoaște adresa IP;
- `Public static InetAddress getLocalhost() throws UnknownHostException` - determină adresa IP a hostului local;
- `Public static InetAddress getAllByName (String host)` - returnează în tablou adresele IP ale hostului current;
- `Public byte [] getAddress()` - returnează adresa IP a obiectului curent în formă de șir;
- `Public String getHostName()` - returnează numele calculatorului căruia îi corespunde obiectul `InetAddress`;
- `Public String getAddress()` – returnează adresa IP căruia îi corespunde obiectul `InetAddress`.

Exemplul 1

Programul ilustrează funcționalitatea acestei clase, se tipărește adresa mașinii locale și apoi într-o buclă se acceptă nume de hosturi și le verifică.

```
import java.net.*;
import java.io.* ;
public class InetExample {
    }
    public static void main (String[] args){
        printLocalAddress () ;
        Reader kbd = new FileReader (FileDescriptor.in) ;
        BufferedReader bufferedKbd =  new BufferedReader (kbd) ;
        try {
            String name ;
            do {
                System.out.print("nume de host sau adresa IP:");
                System.out.flush () ;
                name = bufferedkbd.readLine () ;
                if (name !=null);
                    printRemoteAddress (name);
            } while (name !=null);
            System.out.println ("exit");
        } catch(IOException ex) {
            System.out.println (Input error: ");
            ex.printStackTrace();
        }
    }
}

//static void printLocalAddress()...
```

PROGRAMAREA ÎN REȚEA FOLOSIND SOCKETURI JAVA (2)

```
//static void printRemoteAddress (String name)...,
```

cu procedurile

```
static void printLocalAddress () {
    try {
        InetAddress myself = InetAddress.getLocalHost();
        System.out.println("nume:" + myself.getHostName());
        System.out.println("IP:" + myself.getHostAddress());
        System.out.println("clasa:" + ipClass(myself.getAddress()));
    }
    catch (UnknownHostException ex)
    {
        System.out.println("nu ma gasesc");
        ex.printStackTrace();
    }
}

//static char ipClass (byte [] ip){
int highByte=0xff&ip[0];
return int highByte<128)?'A':(highByte<192)?'B':
(highByte<224)?'C':((highByte <240)?'D':'E');
}

static void printRemote Address (String name) {
    try {
        System.out.println("cauta"+name+"...");
        InetAddress machine = InetAddress.getByName (name);
        System.out.println("nume:" + machine.HostName());
        System.out.println("IP:" + machine.getHostAddress());
        System.out.println("clasa:" + ipClassmachine.getAddress());
    }
    catch (UnknownHostException ex {
        system.out.println {"nu gaseste hostul" + name);
    }
}
```

Clasa URL oferă facilități necesare manipulării locatoarelor universali de resurse URL. Forma completă a unui asemenea locator este:

Protocol://nume_calculator:port/nume_cale#ref

Numele calculatorului poate fi în format simbolic sau adresă Internet, portul se specifică dacă nu este folosit cel standard iar ref nu este parte a URL ci referă o parte din documentul specificat funcție de tipul resursei.

Constructorii clasei sunt:

- `Public URL (String protocol, String host, int port, String file)` throws `MalformedURLException` – creează un obiect URL;
- `Public URL (String protocol, String host, String file)` throws `MalformedURLException` - creează un nou obiect URL absolut pentru care se folosește portul standard;

- `Public URL (URL context, String spec) throws MalformedURLException` - creează un URL bazat pe interpretarea specificării `spec` corespunzător contextului `dat.`, dacă argumentul `context` este `null`, iar `spec` este doar o specificare parțială componentele lipsă vor fi preluate din context;
- `Public URL (String spec) throws MalformedURLException` – creează un obiect URL din reprezentarea string dată.

2.3. Socketuri TCP

2.3.1. Clasa Socket

Clasa *Socket* (din pachetul `java.net`) oferă posibilitatea comunicării folosind socketurile, iar clasa *ServerSocket* implementează un socket folosit în servere pentru a aștepta conexiuni de la clienți.

Un socket în Java este reprezentarea unei conexiuni TCP, folosind această clasă un client poate crea canale de comunicație de tip streamuri de date. După stabilirea conexiunii, hosturile local, respectiv remote pot comunica folosind o conexiune tip full duplex.

Constructorii cei mai utilizați ai acestei clase sunt:

- `Protected Socket ()` - creează un socket neconectat, care poate fi ulterior atașat la o conexiune;
- `Socket (String host, int port) throws UnknownHostException, IOException` - creează un socket TCP și încearcă conectarea la portul hostului specificat prin *nume*;
- `Socket (InetAddress address, int port ) throws IOException` – creează un socket TCP și-l conectează la portul specificat al hostului identificat prin *adresă*;
- `Socket (String host, int port, InetAddress localAddr, int localPort) throws IOException` și `Socket (InetAddress address, int port, InetAddress localAddr, int local Port) throws IOException` - creează socketul TCP, îl leagă la hostul (adresa) și portul local și se conectează la hostul remote. Acesta este constructorul care permite alegerea manuală a interfeței care va fi conectată.

Metodele clasei permit identificarea hostului remote, a porturilor utilizate și extragerea streamurilor utilizate în comunicația bidirecțională. Comunicația folosind socketurile în Java presupune crearea unui *Socket* apoi utilizarea metodelor `getInputStream()` și `getOutputStream()` pentru a obține streamurile necesare comunicației. Alte metode definite în această clasă sunt: `getInetAddress()`, `getPort()`, `toString()`.

Excepțiile sunt cele cauzate de imposibilitatea legării la hostul local sau remote, refuzarea unei conexiuni datorită inexistenței serverului care să asculte pe un anumit port sau imposibilitatea atingerii hostului remote.

Exemplul 2

Descărcarea paginilor web folosind socketuri

Programul conține o buclă de așteptare a unui URL pentru care se creează un obiect care așteaptă conectarea la serverul web pentru a descărca și afișa pagina cerută. Trebuie remarcat faptul că același rezultat se obține utilizând o implementare folosind clasa URL.

```
import java.net. * ;
import java.io. * ;

public class GrabPage {
    //public Grabpage (String textURL)throws IOException....
    //public void grab () throws IOException...
    //public static void main (String[] args ) throws IOException...

    public GrabPage (String textURL) throws IOException {
        dissect (textURL);
    }
    //protected void dissect (String textURL) throws MalformedURLException....

    .....

    protected String host, file ;
    protected int port ;
    protected void dissect (String textURL)throws MalformedURLException {
        URL url = new URL (textURL) ;
        host = url.getHost () ;
        port = url.getPort () ;
        if (port == -1)
            port = 80;
        file = url.getFile() ;
    }

    public void grab () throw IOException {
        connect () ;
        try {
            fetch ();
        } finally {
            disconnect () ;
        }
    }
    //protected void connect ()( throws IOException.....
    //protected void fetch() throws IOException...
    //protected void disconnect () throws IOException...

    .....
    protected Writer writer;
    protected BufferedReader reader ;
    protected void connect ()throws IOException {
        Socket socket = new socket (host, port) ;
```

```
OutputStream out = socket.getOutputStream() ;
writer new OutputStreamWriter (out,"latin1") ;
InputStream in=socket.getInputStream () ;
Reader reader = new InputStreamReader (in, "latin1") ;
this.reader = new BufferedReader (reader) ;
```

```
protected void fetch () throws IOException{
    writer.write( "GET" + file +"HTTP/1.0\r\n\n");
    writer.flush();
    PrintWriter console = new PrintWriter (System.out);
    String input ;
    while ((input = reader.readLine ())!=null)
        console.println (input);
    console.flush() ;
}
```

```
protected void disconnect() throws IOException{
    reader.close ();
}
```

si deasemenea clientul care asteapta preluarea unui URL, la EOF se va iesi din bucla folosind break.

```
public static void main(String[]args throws IOException{
    Reader kbd = new FileReader (FileDescriptor.in);
    BufferedReader bufferedKbd = new BufferedReader (kbd) ;
    while true( {
        String textURL ;
        System.out.print ("introdu URL: ");
        System.out.flush () ;
        if textURL = bufferedKbd.readline () == null)
            Break ;
        try {
            GrabPage grabPage = new GrabPage (textURL);
            GrabPage.grab ();
        }catch {IOException ex) {
            ex.printStackTrace () ;
            continue;
        }

        System.out.println ( "OK");
    }
    System.out.println ("exit");
}
```

2.3.2. Clasa ServerSocket

Această clasă oferă funcționalitatea mecanismului prin care un server acceptă conexiuni de la clienți, ea permite crearea unei conexiuni socket pentru fiecare client apoi serverul va gestiona aceste conexiuni extrăgând streamuri de intrare și ieșire pentru a comunica cu clientul, după ce a fost creată o conexiune de acest tip, va putea fi utilizat un socket pentru a transfera date.

Ciclul de viață al unui server clasic conține următoarele etape: crearea unui nou socket- server care va asculta conexiuni folosind metoda `accept()` (metoda care va returna un obiect de tip socket), efectuarea de transferuri de streamuri (in, out sau in și out), clientul și serverul vor continua interacțiunea respectând un anumit protocol, apoi se va închide conexiunea de către unul din capetele comunicante, și în final se va aștepta după o nouă conexiune.

Proiectarea unui server complex presupune utilizarea threadurilor astfel încât serverul să fie capabil să proceseze cât de repede posibil orice nouă conexiune.

Constructorii clasei pentru crearea de noi socketuri sunt:

- `ServerSocket(int port) throws IOException, BindException` – creează un socket la portul specificat, valoarea zero semnifică orice port disponibil;
- `ServerSocket(int port, int backlog) throws IOException, BindException` – creează un socket la portul specificat și cu numărul de cereri de conectare care pot fi păstrate în așteptare (înainte de a refuza alte cereri, implicit 50 cereri);
- `ServerSocket (int port, int backlog, InetAddress bindAddr) throws IOException` – socketul creat se leagă la adresa de IP specificată. Este util pentru sistemele cu mai multe adrese de IP, permițând alegerea acesteia.

Complementar există definiți constructori care pot fi utilizați pentru a crea propriile implementări de socketuri, eventual utilizând servere de proxy sau diverse protocoale de securitate.

Metodele clasei:

- `accept() throws IOException` - metoda este utilizată la acceptarea unei conexiuni. Este soluția de comunicație blocantă deoarece serverul va fi oprit până la conectarea clientului;
- `getInetAddress()` – returnează adresa utilizată de server (a hostului local, pentru sisteme multihomed nu se poate previziona care adresă va fi returnată);
- `getLocalPort()` – permite ascultarea unui port nespecificat, această metodă permite aflarea acestui port.

Multe din protocoale folosesc presupunerea că aceste socketuri se închid singure după un anumit număr de schimburi de mesaje deoarece pentru anumite programe care pot rula la infinit este necesar ca socketurile să fie închise.

`Close() throws IOException` – eliberează portul pentru alte programe care îl pot utiliza, prin închiderea conexiunii.

După închiderea unui socket, numărul de port și adresa locală sunt încă accesibile folosind metodele `GetInetAddress()`, `getLocalPort()`, `getPort()`.

Setarea opțiunilor specifice unui socket - (set)getSoTimeout(int timeout) - sunt utilizate în protocoale complexe care necesită conexiuni multiple între client și server (valoarea implicită este 0, de obicei setarea parametrului presupune specificarea înaintea metodei accept() a valorii de timeout). Alți parametri care sunt necesari în cazul unor protocoale complexe sunt TCP_Nodelay – asigură transmiterea pachetelor cât de repede posibil sau SO_Linger – specifică modul de tratare a datagramelor transmise după închiderea unui socket.

Exemplul 3

Implementarea unui server de echo care acceptă conexiuni și apoi retransmite în ecou ceea ce a recepționat.

```
import java.net.*
import java.io.*
Public class STServer{
    Public static void main String[] args} throws IOException...
}

public static void main (String[] args)throws IOException
    if args.length!=1)
        throw new Illegal ArgumentException ("exceptie");
    Socket client = accept (Integer.parseInt(args[0]));
    try{
        InputStream in = client.getInputStream();
        OuputStream out = client.getOutputStream();
        out.write ("esti conectat la server\r\n".getBytes ("latin1")
        int x;
        While ((x=in.read())> -1)
            Out.write (x);
    }finally{
        System.out.println ("Inchide");
        Client.close();
    }
}
// static Socket accept (int port) throws IOException...

Static Socket accept (int port) throws IOException {
    System.out.println ("porneste pe portul"+port);
    Server.Socket server = new ServerSocket (port);
    System.out.println ("asteapta");
    Socket client = server.accept();
    System.out.println ("acceptata de la ..."+client.getInetAddress());
    server.close();
    return client;
}
```

Testarea serverului se poate realiza utilizand adresa de loopback.127.0.0.1

Exemplul 4

Server multithread care conține un thread ce va deservi acceptarea conexiunilor, și un alt thread care va deservi conexiunile existente.

```
import java.net.*
import java.io.*

Public class MTEerver extends Thread{
//MTEchoServer(Socket socket)....
//public void run()...
//public static void main (String[] args) throws IOException...
}

protected Socket socket
MTEchoServer (Socket socket){
this.socket=socket;
public void run(){
try {

InputStream in = socket.getInputStream();
OutputStream out = socket.getOutputStream();
out.write ("esti conectat la server multithread\r\n".getBytes ("latin1"))
byte[] buffer = new byte [1024];
int read;
while ((read = in.read (buffer))>=0)
out.write (buffer,0,read);
}catch (IOException ex){
ex.printStackTrace ();
}finally{
try{
socket.close();
}catch (IOException ignored){
}
}
}

public static void main (String[]args)throws IOException{
if (args.length!=1)
throw new IllegalArgumentException ("MTEchoServer<>");
System.out.println ("Porneste ps portul"+args{});
ServerSocket server = new ServerSocket(Integer.parseInt (args{}));
While (true){
Socket client= server.accept();
MTEchoServer echo = new MTEchoServer(client);
Echo.start();
}
}
```

Exemplul 5. Implementarea unui server neblocaant NB, folosind metoda available().


```
import java.net.*;
import java.io.*;
Public class NBServer{
    //public static void main(String[] args) throws IOException...
}

static InputStream in0, in1;
static OutputStream out0, out1;
public static void main (String[] args ) throws IOException{
    if(args.length !=1)
        throw new IllegalArgumentException (Server: port");
    try{
        accept (Integer.parseInt (args[0]));
        int x0, x1;
        while (((x0 = readNB(in0))!=-1&&
                ((x1=readNB (in1))!=-1)){
            if (x0>=0)
                out1.write (x0);
            if (x1>=0
                out0.write (x1);
        }
    }finally{
        System.out.println ("inchide");
        close(out0);
        close(out1);
    }
}

static void close (OutputStream out){
    if (out !=null){
        try{
            out.close();
        }catch(IOException ignores{
        }
    }
}

//static void accept (int port throws IOException...
//static int (readNB *InputStream in)...

static void accept(int port)throws IOException{
    System.out.println" se porneste pe portul" +port);
    ServerSocket server = new ServerSocket(port);
    try{
        System.out.println "asteapta...");
        Socket client0 =server.accept ();
        System.out.println "de la clientul "+client0.getInetAddress());
        in0 = client0.getInputStream();
        out0= client0.getOutputStream();
        out0.write ("hello, asteapta te rog\r\n".getBytes ("latin1"));
        System.out.println ("asteapta...");
        Socket.client1 = server.accept ();
        System.out.println (accepta de la"+client1.getInetAddress());
        in1 = client.getInputStream();
        out1= client1.getOutputStream();
```

```
        out1.write ("hello\r\n".getBytes("latin1"));
        out1.write ("porneste\r\n".getBytes("latin1"));
    }finally{
        server.close();
    }
}

static int readNB (InputStream in) throws IOException{
    if (in.available()>0)
        return in.read();
    else
        return -2;
}
```

2.4. Socketuri datagrame

În pachetul Java.net există clasele *DatagramSocket*, *DatagramPacket* și *MulticastSocket* care oferă suport pentru programarea transmisiei de date folosind datagrame. Fiecare datagramă conține un header și o zonă de date. Headerul cuprinde informațiile de adresare (portul și adresa sursă și destinație) și alte informații legate de asigurarea transmisiei. Datagramele au lungime fixă de aceea este nevoie uneori să fie împărțite în pachete și refăcute la destinație. Transmisia de datagrame se realizează fără stabilirea prealabilă a unei conexiuni, pachetele transmise nu trebuie recepționate în aceeași ordine și deasemenea un socket de tip datagramă poate recepționa date de la mai multe hosturi diferite.

2.4.1 Clasa DatagramPacket

Clasa *DatagramPacket* organizează datele în pachete tip UDP.

Constructorii clasei *DatagramPacket* sunt:

- *DatagramPacket* (byte buffer [], int length) - unde, buffer reprezintă un tablou care memorează pachetul, iar length reprezintă numărul maxim de bytes care pot fi citiți în buffer;
- *DatagramPacket* (byte buffer [], int length, InetAddress address, int port) – pachetul va fi livrat la adresa de host și portul specificate. Este necesar ca la celălalt capăt al comunicației să existe un server UDP care să asculte conexiuni. Volumul de date care poate fi plasat într-un pachet depinde de tipul protocolului, și de tipul tehnologiei utilizate.

Metodele acestei clase sunt:

- *GetAddress()* – returnează obiectul *InetAddress* reprezentând adresa hostului remote;
- *GetPort()* - returnează portul datagramei pe mașina remote;
- *GetLength()* - returnează lungimea datagramei (număr de bytes);

- `GetData()` – returnează un tablou care conține datele transferate folosind datagrame.

2.4.2 Clasa DatagramSocket

Această clasă permite transmiterea și recepția datagramelor. Constructorii clasei sunt:

- `DatagramSocket()` throws `SocketException` - creează un socket datagramă la un port aleator ales (anonim);
- `DatagramSocket (int port )` throws `SocketException` - creează un socket care ascultă la un port specificat;
- `DatagramSocket (int port, InetAddress local)` throws `SocketException` – socketul creat este și legat la o adresă locală.

2.4.3 Transmiterea și recepția datagramelor

După crearea unui `DatagramPacket` și construirea unui `DatagramSocket` pachetul acesta va fi transferat folosind metoda `send()`, respectiv recepționat folosind metoda `receive()`.

```
Send (DatagramPacket dp) throws IOException  
Receive (DatagramPacket dp) throws IOException
```

Secvența următoare reprezintă utilizarea datagramelor Java pentru a recepționa un pachet de date. Se creează un socket de tip datagramă `DatagramSocket`, la un port specificat.

Exemplul 5

```
Socket = new DatagramSocket (...);  
...  
byte[] buffer = new byte [65508]  
datagramPacket packet = new DatagramPacket (buffer,buffer.length);  
socket.receive(packet);  
.....  
socket.close()  
  
.....  
InetAddress address = packet.getAddress();  
Int fromPort = packet.getPort();  
Int length = packet.getLength();  
Byte[] data = packet.getData();
```

La construirea unui pachet datagramă trebuie să se specifice adresa IP și portul destinație pentru ca acesta să poată fi transmis. Exemplul următor ilustrează acest aspect astfel:

```
DatagramSocket packet = new DatagramSocket ();
```

```
DatagramPacket packet = newDatagramPacket  
(data,data.length,InetAddress.getByName (www.utcluj.ro,1728) );  
Socket.send(packet) ;  
Socket.close();
```

2.4.4 Socketuri multicast. Clasa MulticastSocket

Transmisia (recepția) multicast oferă un set de avantaje de loc de neglijat între care simplificarea proiectării și reducerea consumului de lățime de bandă pentru un set de aplicații a căror topologie necesită arbori de acoperire (din domeniul procesării masiv paralele, a serviciilor de nume, a serviciilor de directoare distribuite, a mecanismelor de replicare a bazelor de date, etc.). Pentru a recepționa date multicast de la un host remote, după crearea unui socket multicast este necesară asocierea la un grup de multicast apoi va putea fi stabilită conexiunea pentru a transmite și recepționa date. Asocierea la grup anunță ruterele aflate în calea către serverul care oferă serviciul să transmită datele în mod multicast, hostul fiind responsabil cu adresarea pachetelor în mod multicast. Atașarea la grup determină comunicația similară celei utilizate la transferul de datagrame, trebuie remarcat faptul că doar pentru a transmite în mod multicast nu este necesară atașarea la un grup multicast. Ceea ce diferențiază totuși comunicarea multicast de cea unicast o constituie specificarea și gestionarea câmpului TTL (Time to live) câmp ce permite evitarea buclelor de rutare pentru topologii arborescente complexe.

Adresele de tip multicast sunt adrese de clasă D în gama de adrese 224.0.0.0 – 239.255.255.255, domeniu structurat pe diferite tipuri de adrese multicast rezervate sau nu.

Constructorii clasei *MulticastSocket()* utilizați în procesarea datagramelor multicast sunt:

- *MulticastSocket()* throws *SocketException* – crează un socket legat la portul anonim (orice port asignat de sistem) și util pentru clienții ce nu necesită specificarea unui port;
- *MulticastSocket (int port)* throws *SocketException*

Metodele clasei necesare comunicării cu un grup multicast sunt:

- *JoinGroup(InetAddress mcastaddr)* throws *SocketException*;
- *LeaveGroup(InetAddress mcastaddr)* throws *SocketException*;
- *Send (DatagramPacket dp, byte ttl)* throws *IOException, SocketException*;
- *Set (Get)Interface (InetAddress interface)* throws *SocketException* – pentru alegerea interfeței utilizate pentru transmisia multicast în hosturile multihomed.

Pentru recepția de pachete multicast poate fi folosită metoda *receive* a claselor *DatagramSocket*.

Exemplul următor își propune ilustrarea modului de utilizare în recepție, respectiv transmisie a pachetelor de tip multicast, datorită faptului că există multe similitudini cu transmisia datagramelor. Transmiterea unui pachet unui grup multicast presupune specificarea valorii unui câmp special TTL (Time to live, a cărei valoare implicită este

255, alte valori uzuale: localhost-0, subrețea- 1, altă rețea LAN: 16) în conținutul datagramei, cu scopul de a preveni bucle infinite de rutare a datagramelor în rețea, controlînd astfel cât de departe sunt transferate pachetele.

Transmisie:

```
// byte[] data
//InetAddress multicastGroup
//int multicastPort

MulticastSocket socket = new MulticastSocket() ;
DatagramPacket packet = new DatagramPacket (data, data.length,
multicastGroup, multicastPort) ;
Socket.send (packet, (byte) 64 ) ;
Socket.close () ;
```

Corespunzător, secvența pentru recepția unui pachet în mod multicast este:

```
// InetAddress multicastGroup
// int multicastPort

MulticastSocket socket = new MulticastSocket(multicastPort) ;
socket.joinGroup (multicastGroup);
byte[] buffer = new byte [65508] ;
DatagramPacket packet = new DatagramPacket () ;
socket.receive(packet);
InetAddress fromAddr = packet.getAddress() ;
int fromPort = packet.getPort();
int length = packet.getLength() ;
byte [] data = packet.getData() ;
....
socket.leaveGroup (multicastGroup) ;
socket.close () ;
```

3. Desfășurarea lucrării

3.1. Proiectați o aplicație Java care să implementeze comanda finger. Clientul se conectează la server la portul 79 și transmite o linie cu o interogare terminată cu perechea CR_LF la care serverul răspunde.

3.2. Proiectați o aplicație care să implementeze comunicația între un client și un server respectând specificațiile: programul server ascultă rețeaua și așteaptă să primească o cerere de la un client. Dacă cererea este recepționată fără conflicte, serverul procesează cererea și închide conexiunea. Clientul folosește o adresă pentru a solicita o conexiune unui calculator în rețea (în mod uzual solicită un serviciu unui server), apoi se deconectează din rețea.

3.3. Propuneți o soluție de server neblocant care să utilizeze valori de timeout. Proiectați un server concurent care să genereze pentru fiecare conexiune acceptată un nou fir de execuție.

3.4. Să se proiecteze un program server care cere o parolă pentru autentificarea unui client apoi va trimite în rețea un fișier către programul client. Programul client se va conecta la server, va trimite parola solicitată, va solicita un fișier și apoi va salva fișierul solicitat.

PROGRAMAREA ÎN REȚEA FOLOSIND SOCKETURI JAVA (2)

3.5. Să se implementeze o aplicație care să realizeze comunicarea între doi apleți aflați în aceeași pagină Web. Fiecare applet va conține două butoane unul pentru actualizarea appletului în care se găsește butonul, iar cel de al doilea pentru actualizarea celui alt applet.

3.6. Să se scrie un program care să trimită un mesaj de un byte unei mașini server. Să se calculeze timpul mediu (round trip time) necesar transferului mesajului, considerând repetarea procesului de un număr suficient de ori astfel încât să fie posibilă o estimare medie. Să se modifice dimensiunea mesajului (1kb, 2kb, 4kb, 8kb) și să se calculeze debitul pentru fiecare dimensiune de mesaj. Stabiliți o relație între latență, lățimea de bandă și cerințele unei anumite aplicații.

Notițe